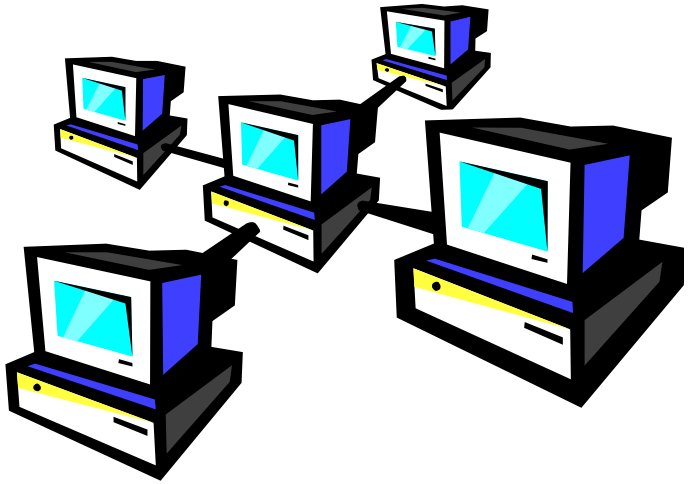
Computer Networks (IN 2511)



Dr. C. Amalraj

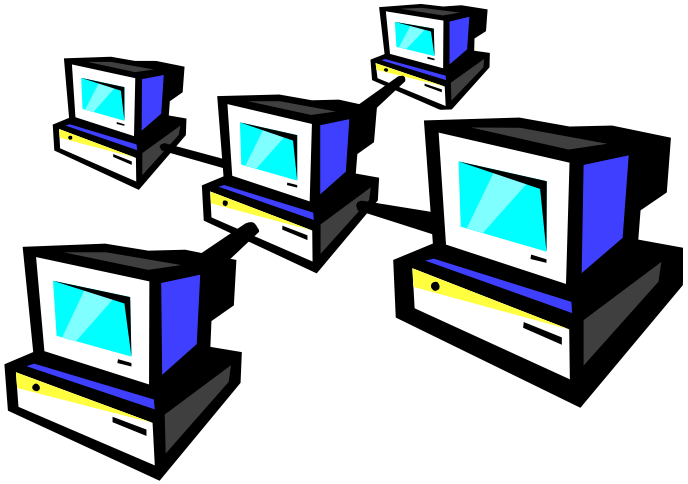
06/06/2025

The University of Moratuwa

amalraj@uom.lk

Lecture 10:

Application Layer



Application Layer

Some network Apps

- E-mail
- Web
- Instant messaging
  - WebSocket, XMPP
- Remote login
- P2P file sharing
- Multi-user network games
- Streaming stored video clips
  - Real-Time Streaming Protocol (RTSP)
  - SIP
  - H.323
- Internet telephone
- Real-time video conference
- Massive parallel computing

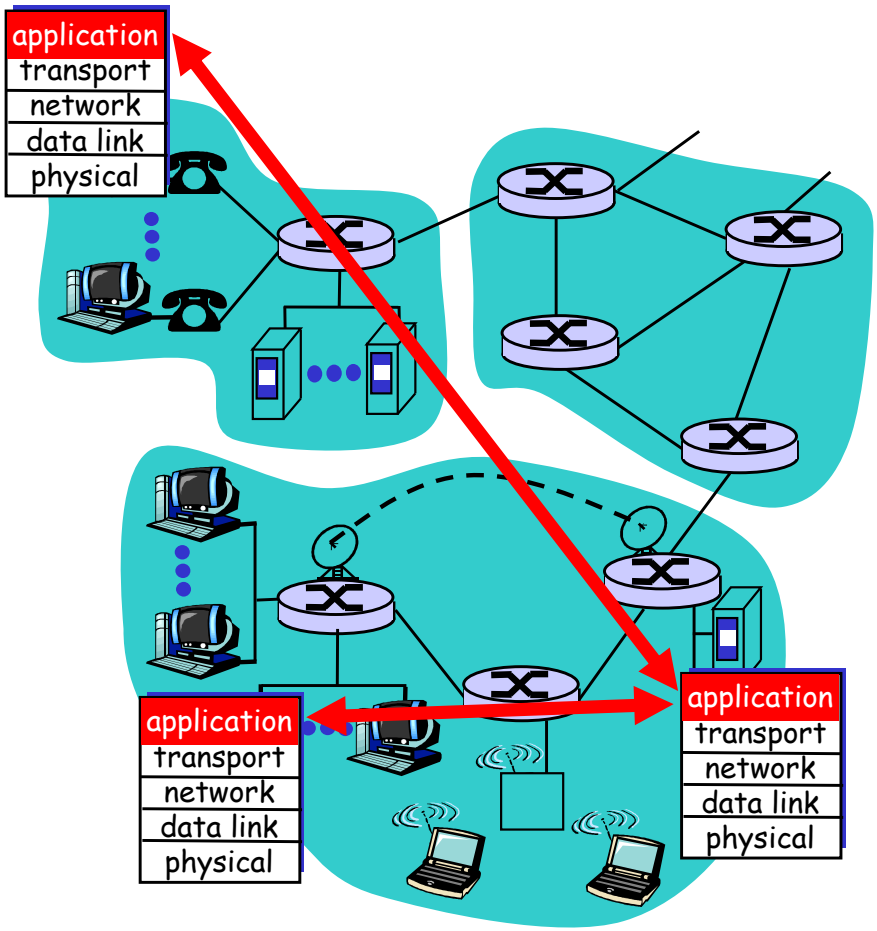
Creating a network app

Write programs that

- run on different end systems and
- communicate over a network.
- e.g., Web: Web server software communicates with browser software

little software written for devices in network core

- network core devices do not run user application code
- application on end systems allows for rapid app development, propagation



Web and HTTP

Web and HTTP

First some jargon

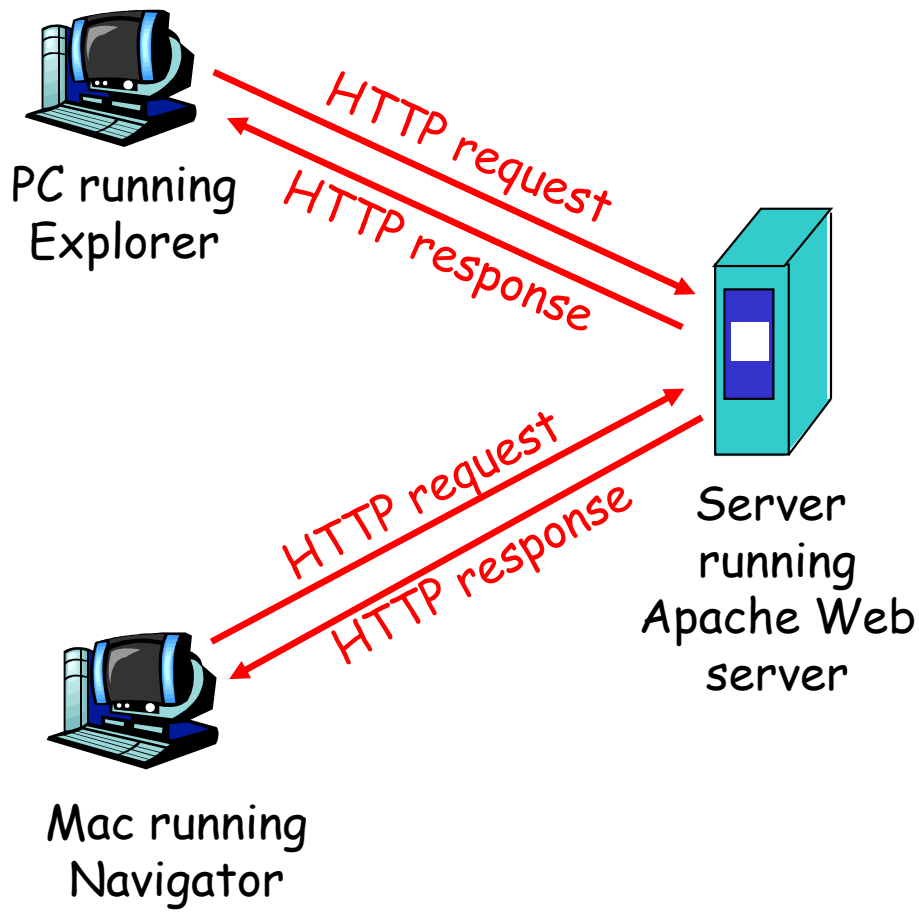
- Web page consists of objects
- Object can be HTML file, JPEG image, Java applet, audio file,…
- Web page consists of base HTML-file which includes several referenced objects
- Each object is addressable by a URL
- Example URL:

$$\underbrace{\texttt{www.someschool.edu}}_{\text{host name}}\underbrace{\texttt{/someDept/pic.gif}}_{\text{path name}}$$

HTTP overview

HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model
  - *client:* browser that requests, receives, "displays" Web objects
  - *server:* Web server sends objects in response to requests
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



HTTP overview (cont.)

Uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains no information about past client requests

aside

Protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

Nonpersistent HTTP

- At most one object is sent over a TCP connection.
- HTTP/1.0 uses nonpersistent HTTP

Persistent HTTP

- Multiple objects can be sent over single TCP connection between client and server.
- HTTP/1.1 uses persistent connections in default mode

Nonpersistent HTTP

Suppose user enters URL

`www.someSchool.edu/someDepartment/home.index`

(contains text, references to 10 jpeg images)

1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80

1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Nonpersistent HTTP (cont.)

4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

time

6. Steps 1-5 repeated for each of 10 jpeg objects

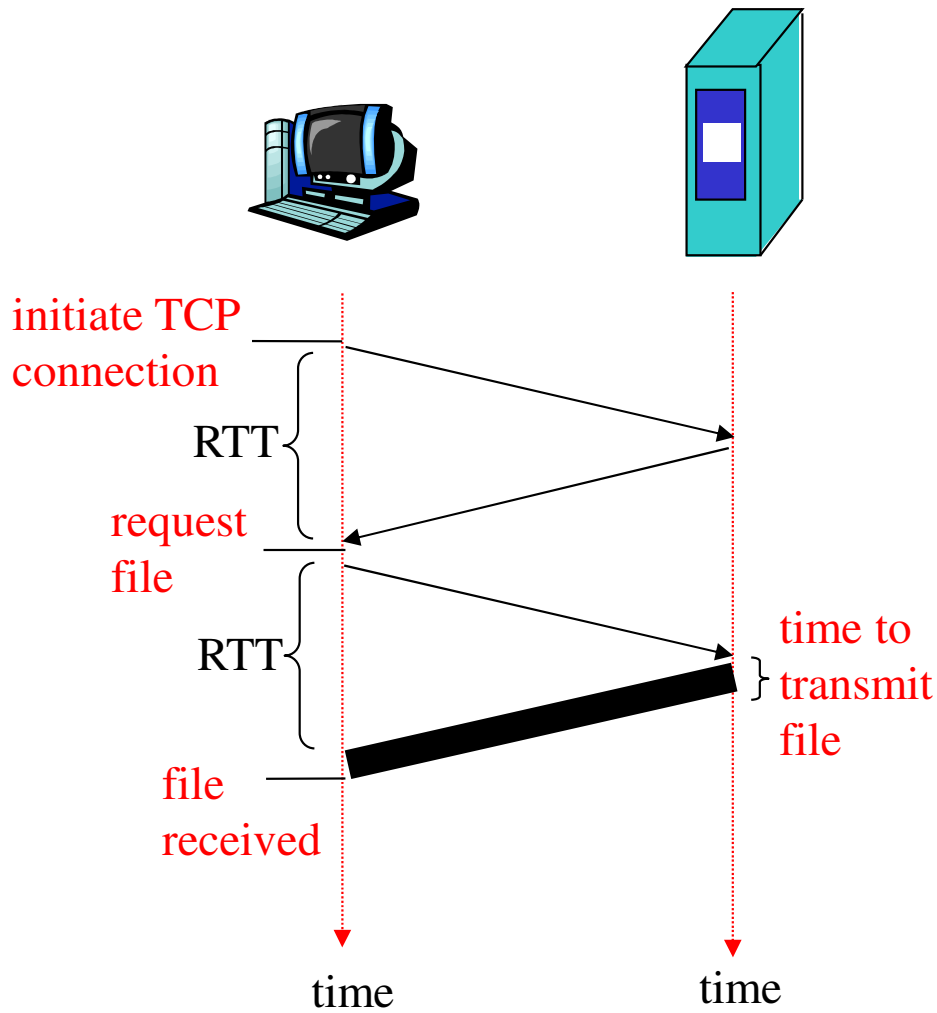
Response time modeling

Definition of RTT: time to send a small packet to travel from client to server and back.

Response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time

total = 2RTT+transmit time



Persistent HTTP

Nonpersistent HTTP issues:

- requires at least 2 RTTs per object
- OS must work and allocate host resources for each TCP connection
- but browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server are sent over connection

Persistent without pipelining:

- client issues new request only when previous response has been received
- at least one RTT for each referenced object

Persistent with pipelining:

- default in HTTP/1.1
- client sends requests as soon as it encounters a referenced object
- may be as little as one RTT for all the referenced objects

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)

request line (GET, POST, HEAD commands)

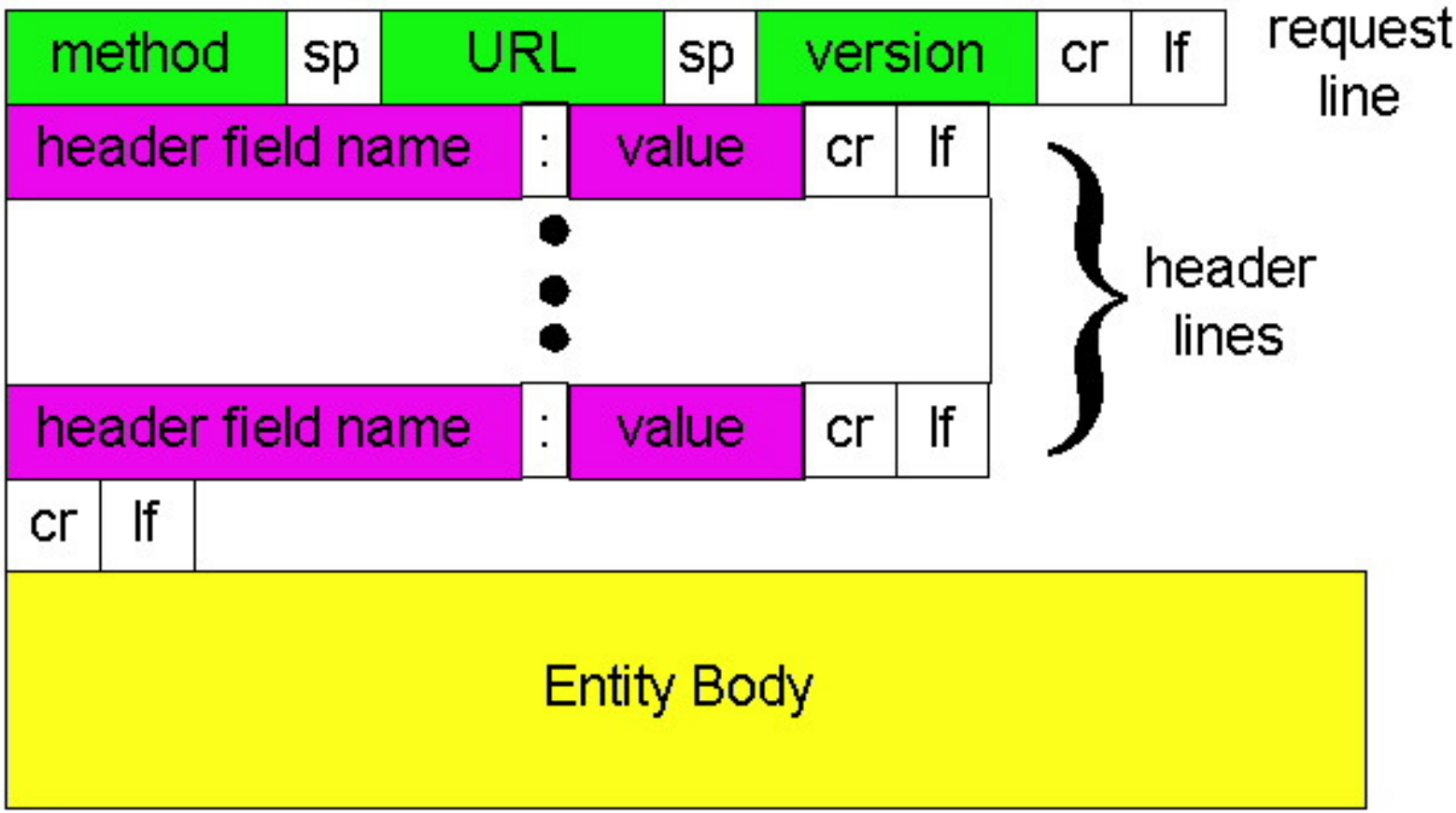
header lines

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language:fr
```

(extra carriage return, line feed)

Carriage return, line feed indicates end of message

HTTP request message: general format



Uploading form input

Post method:

- Web page often includes form input
- Input is uploaded to server in entity body

URL method:

- Uses GET method
- Input is uploaded in URL field of request line:

`www.somesite.com/animalsearch?monkeys&banana`

Method types

HTTP/1.0

- GET
- POST
- HEAD
  - asks server to leave requested object out of response

HTTP/1.1

- GET, POST, HEAD
- PUT
  - uploads file in entity body to path specified in URL field
- DELETE
  - deletes file specified in the URL field

HTTP response message

status line
(protocol
status code
status phrase)

header
lines

data, e.g.,
requested
HTML file

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 …...
Content-Length: 6821
Content-Type: text/html

data data data data data ...
```

HTTP response status codes

In first line in server->client response message.
A few sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (Location:)

400 Bad Request

- request message not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet cis.poly.edu 80
```

Opens TCP connection to port 80 (default HTTP server port) at cis.poly.edu. Anything typed in sent to port 80 at cis.poly.edu

2. Type in a GET HTTP request:

```
GET /~ross/ HTTP/1.1
Host: cis.poly.edu
```

By typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. Look at response message sent by HTTP server!

User-server state: cookies

Many major Web sites use cookies

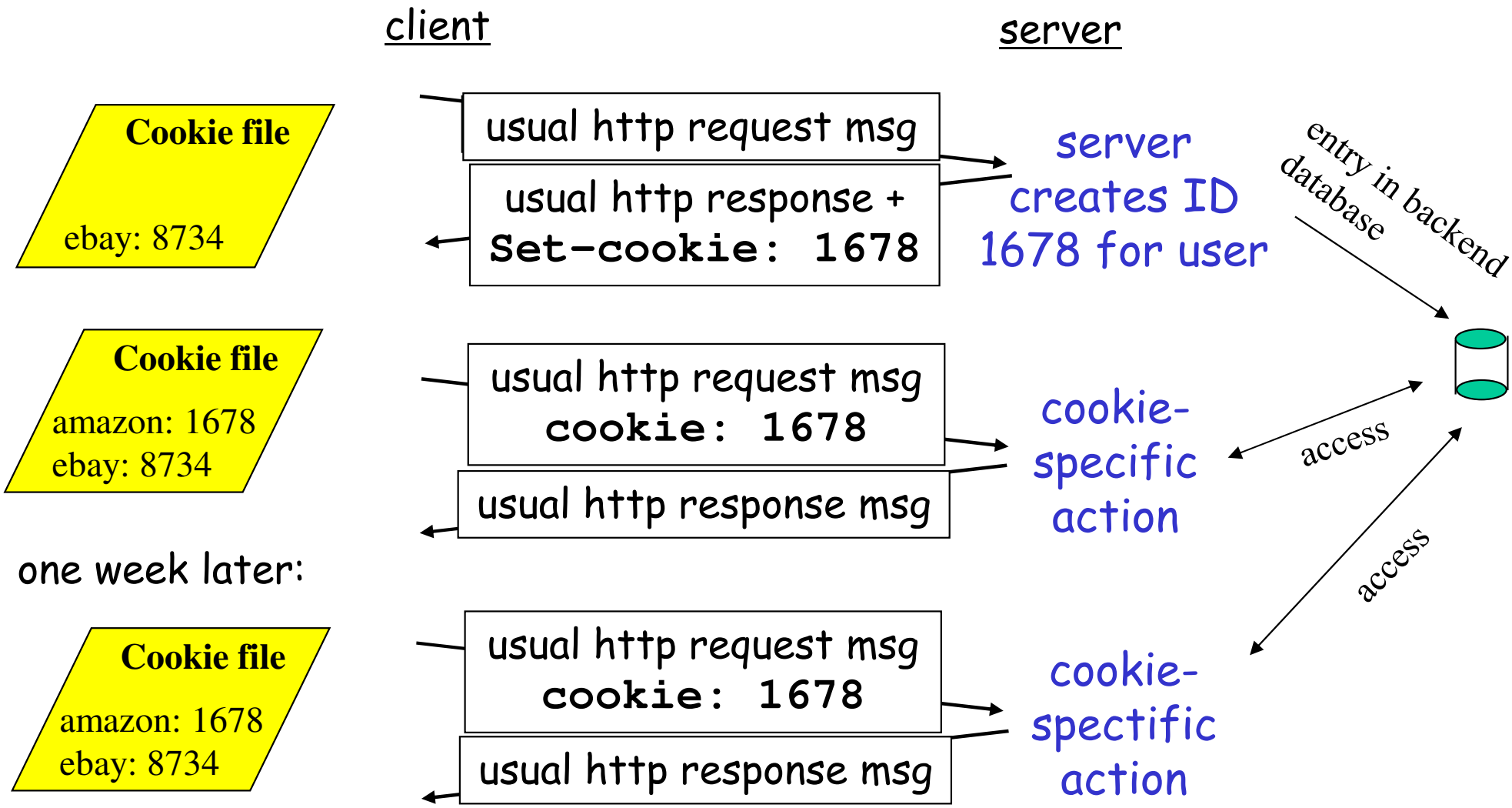
Four components:

1) cookie header line in the HTTP response message
2) cookie header line in HTTP request message
3) cookie file kept on user's host and managed by user's browser
4) back-end database at Web site

Example:

- Susan access Internet always from same PC
- She visits a specific e-commerce site for first time
- When initial HTTP requests arrives at site, site creates a unique ID and creates an entry in backend database for ID

Cookies: keeping "state" (cont.)



Cookies (continued)

What cookies can bring:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

aside

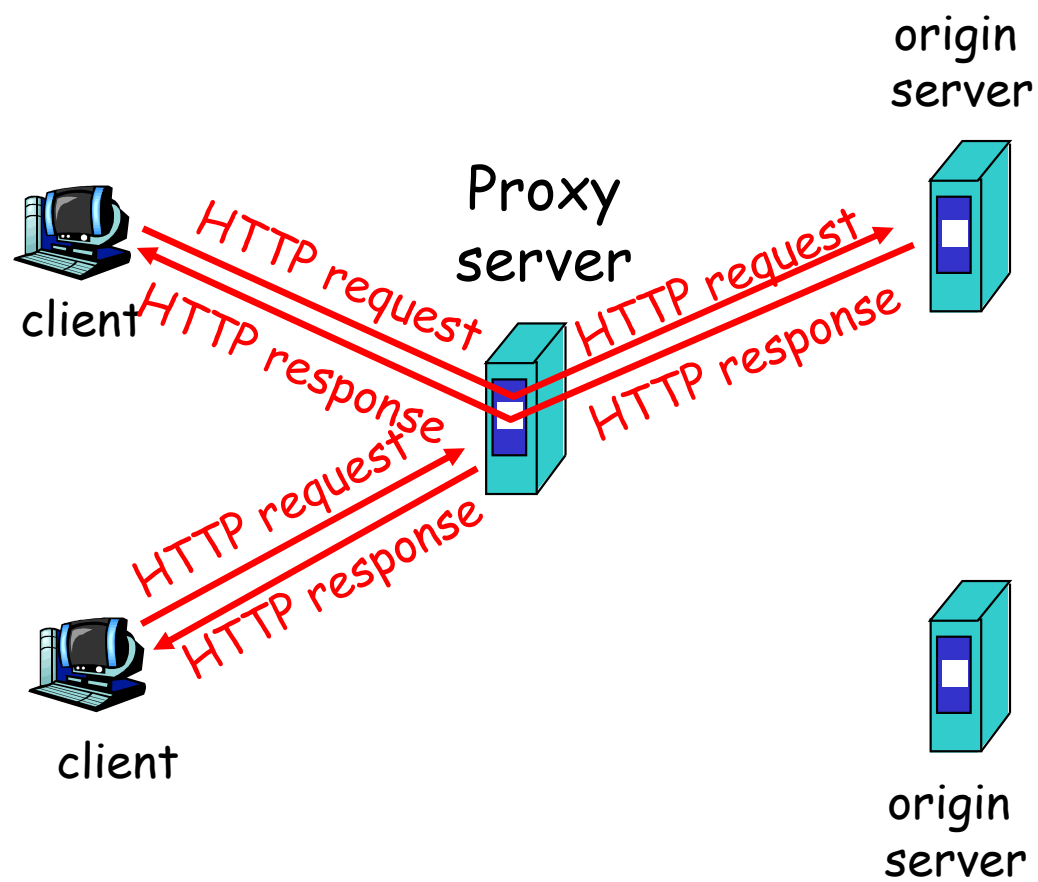
Cookies and privacy:

- cookies permit sites to learn a lot about you
- you may supply name and e-mail to sites
- search engines use redirection & cookies to learn yet more
- advertising companies obtain info across sites

Web caches (proxy server)

Goal: satisfy client request without involving origin server

- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
  - object in cache: cache returns object
  - else cache requests object from origin server, then returns object to client



More about Web caching

- Cache acts as both client and server
- Typically cache is installed by ISP (university, company, residential ISP)

Why Web caching?

- Reduce response time for client request.
- Reduce traffic on an institution's access link.
- Internet dense with caches enables "poor" content providers to effectively deliver content (but so does P2P file sharing)

Caching example

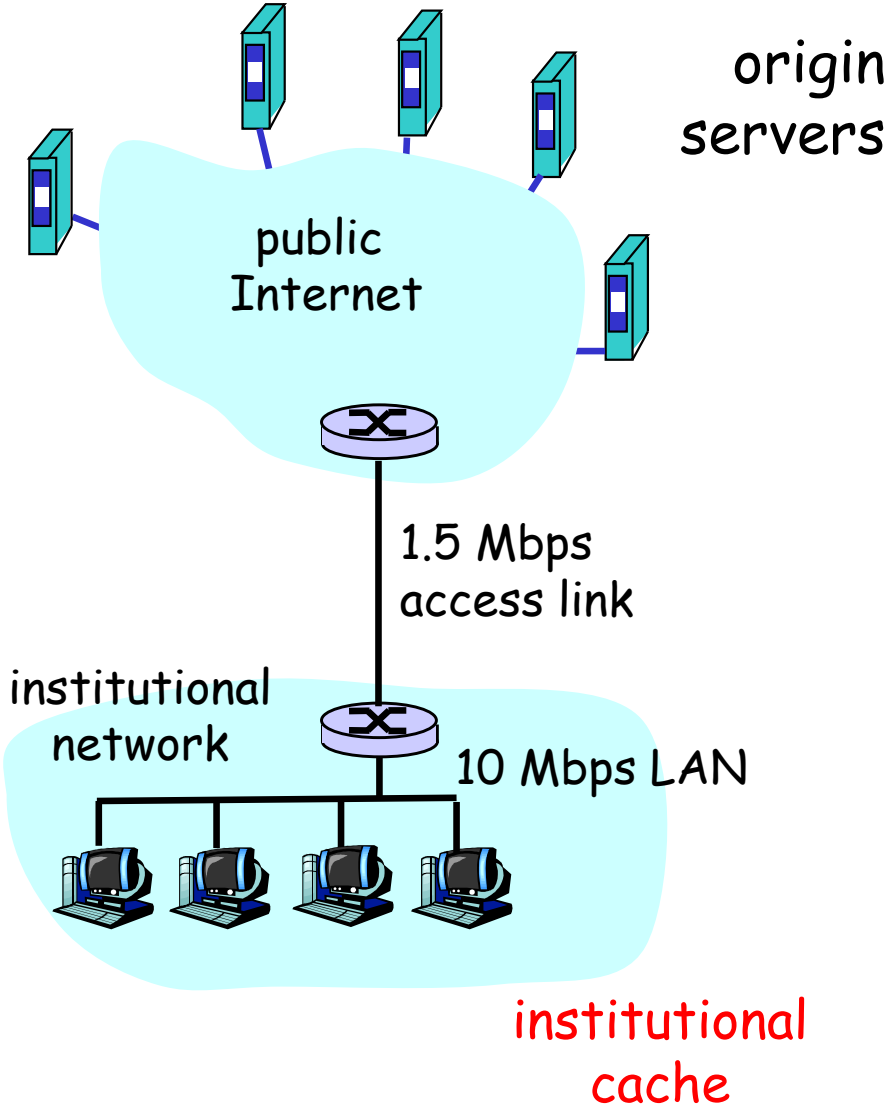
Assumptions

- average object size = 100,000 bits
- avg. request rate from institution's browsers to origin servers = 15/sec
- delay from institutional router to any origin server and back to router = 2 sec

Consequences

- utilization on LAN = 15%
- utilization on access link = 100%
- total delay = Internet delay + access delay + LAN delay

= 2 sec + minutes + milliseconds



Caching example (cont.)

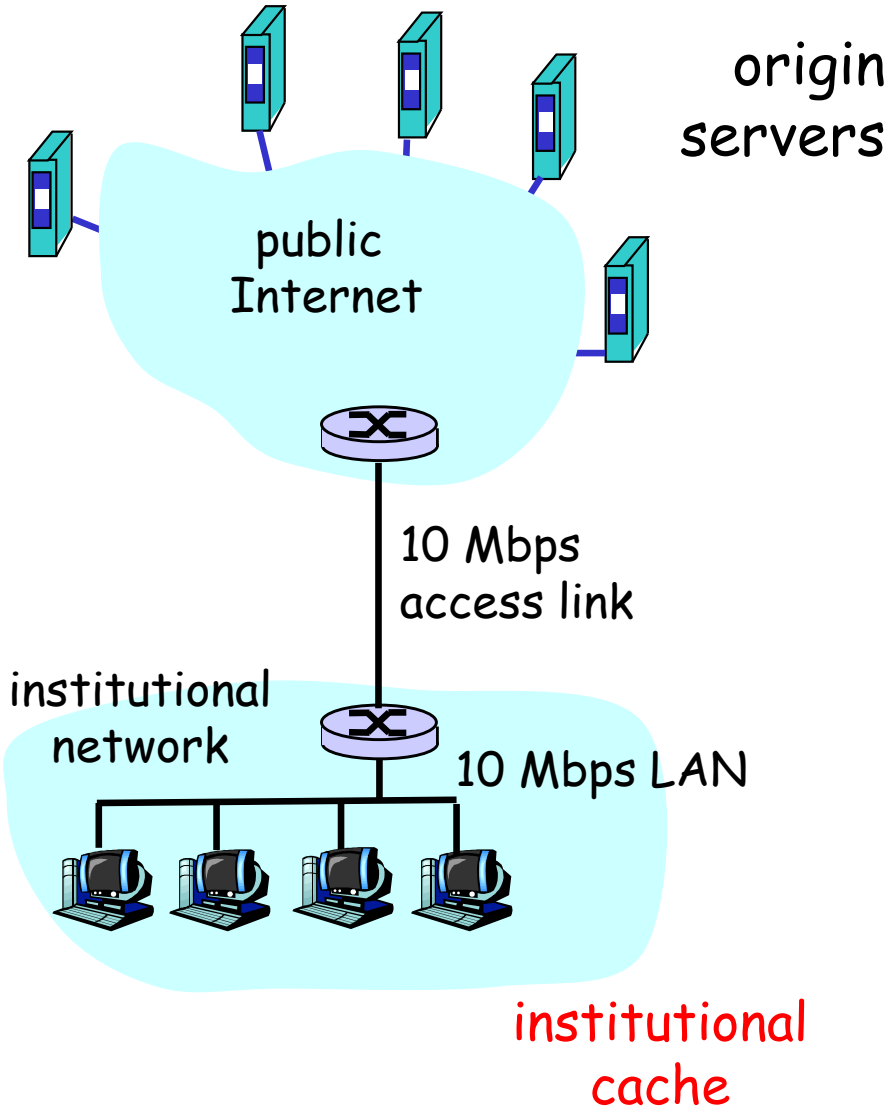
Possible solution

- increase bandwidth of access link to, say, 10 Mbps

Consequences

- utilization on LAN = 15%
- utilization on access link = 15%
- Total delay = Internet delay + access delay + LAN delay

  = 2 sec + msecs + msecs
- often a costly upgrade



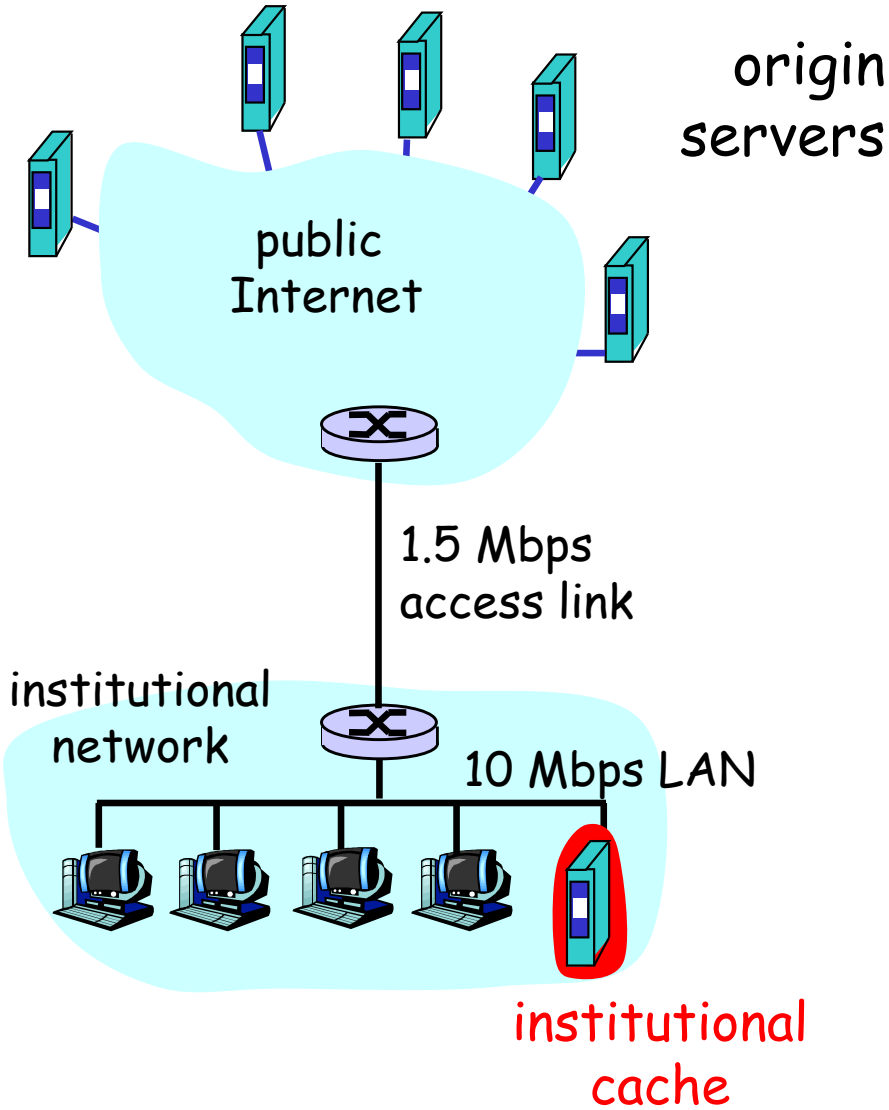
Caching example (cont.)

Install cache

- suppose hit rate is .4

Consequence

- 40% requests will be satisfied almost immediately
- 60% requests satisfied by origin server
- utilization of access link reduced to 60%, resulting in negligible delays (say 10 msec)
- total avg delay = Internet delay + access delay + LAN delay = .6*(2.01) secs + milliseconds < 1.4 secs

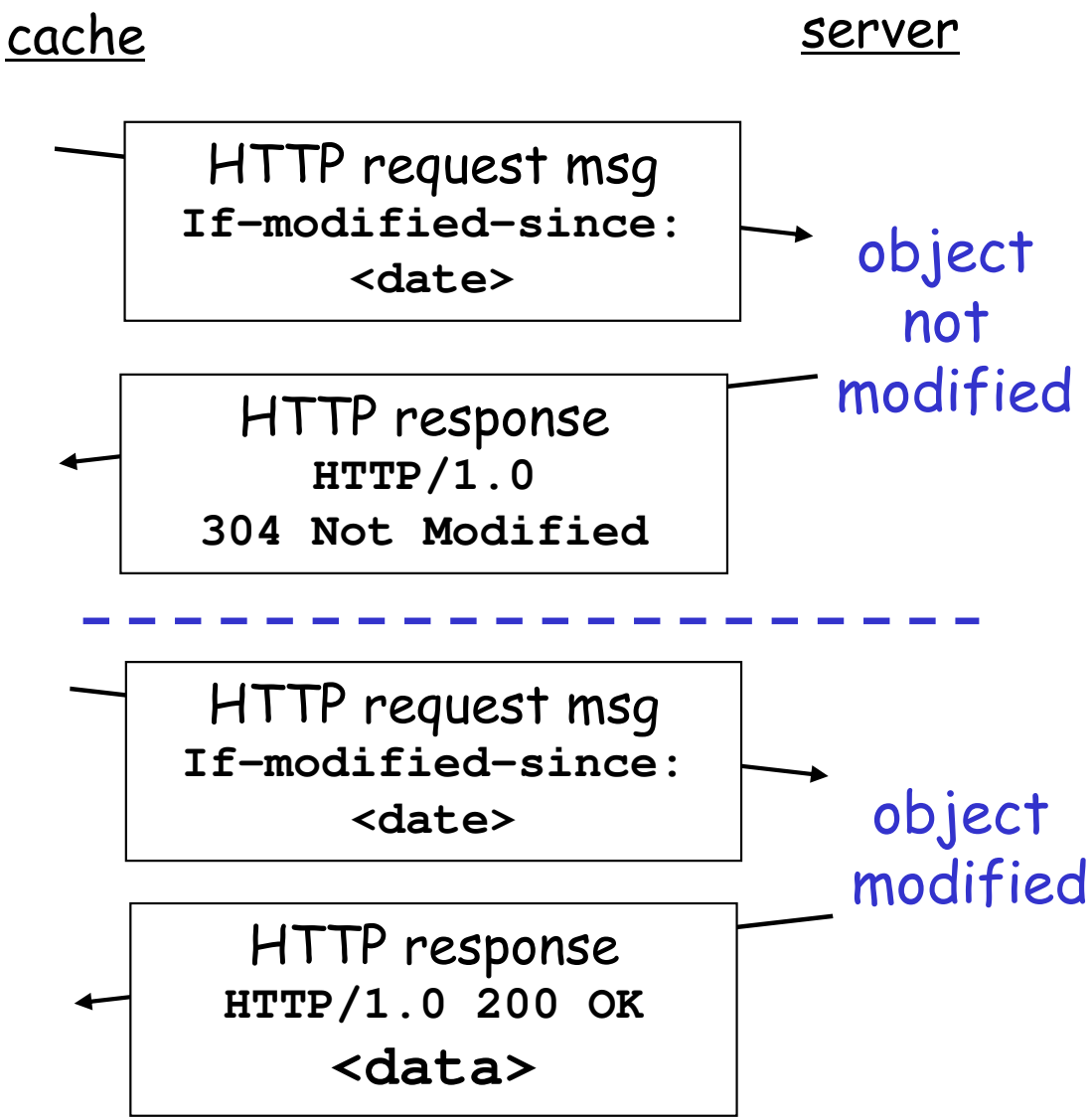


Conditional GET

- Goal: don't send object if cache has up-to-date cached version
- cache: specify date of cached copy in HTTP request

  `If-modified-since: <date>`
- server: response contains no object if cached copy is up-to-date:

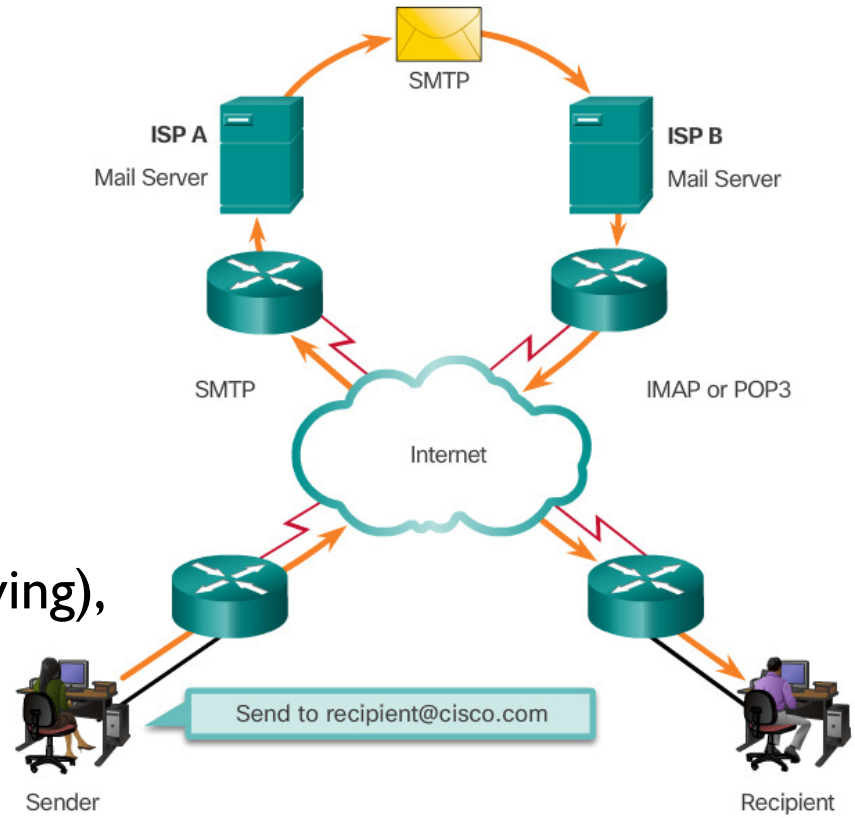
  `HTTP/1.0 304 Not Modified`



Electronic Mail

Email Protocols

- Email is a store-and-forward method of sending, storing, and retrieving electronic messages.
- Email messages are stored in databases on mail servers.
- Email clients communicate with mail servers to send and receive email.
- Mail servers communicate with other mail servers to transport messages from one domain to another.
- Email **clients do not communicate directly** when sending email.
- Email relies on three separate protocols for operation: SMTP (sending),POP (retrieving), IMAP (retrieving).



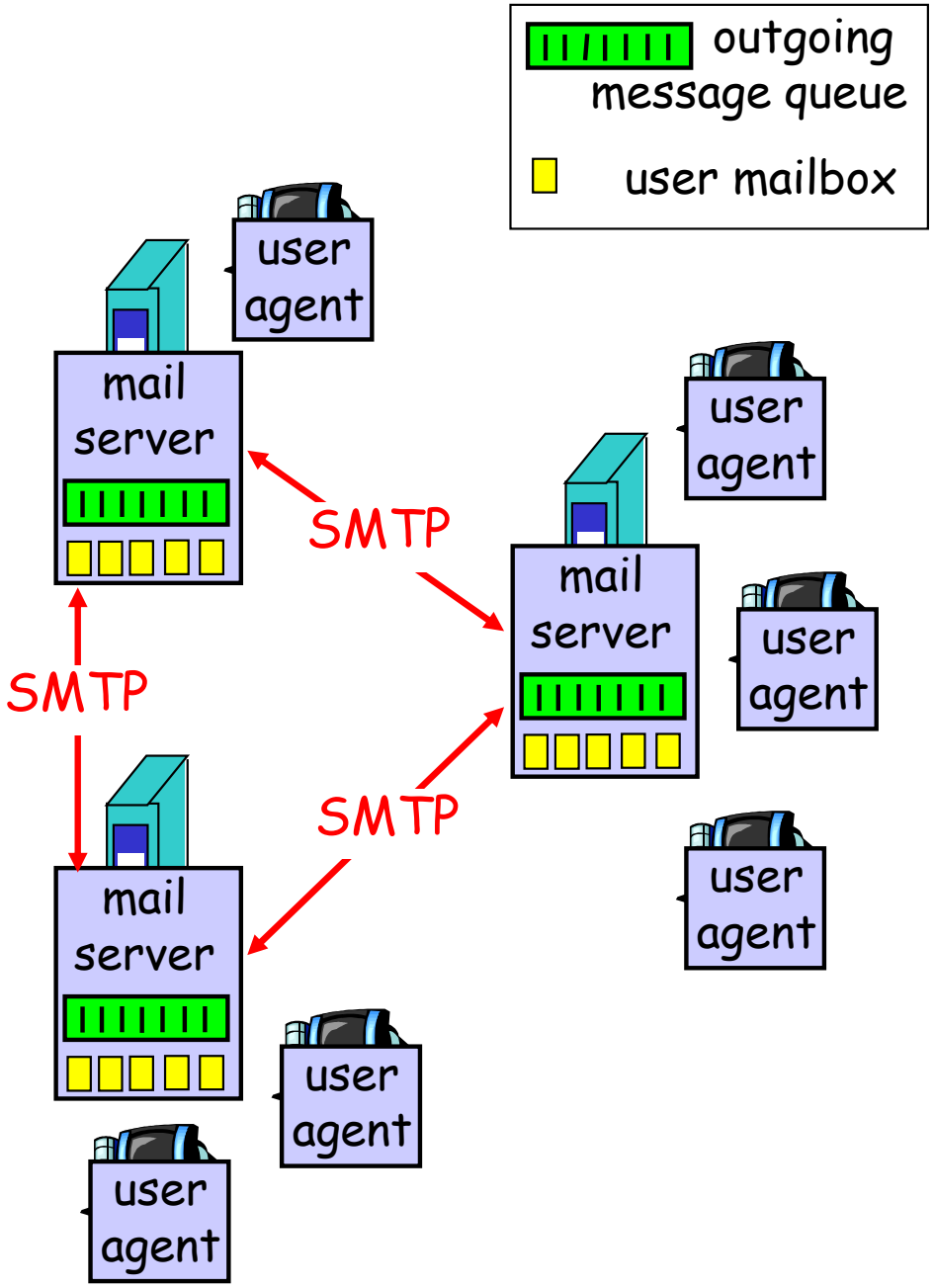
Electronic Mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

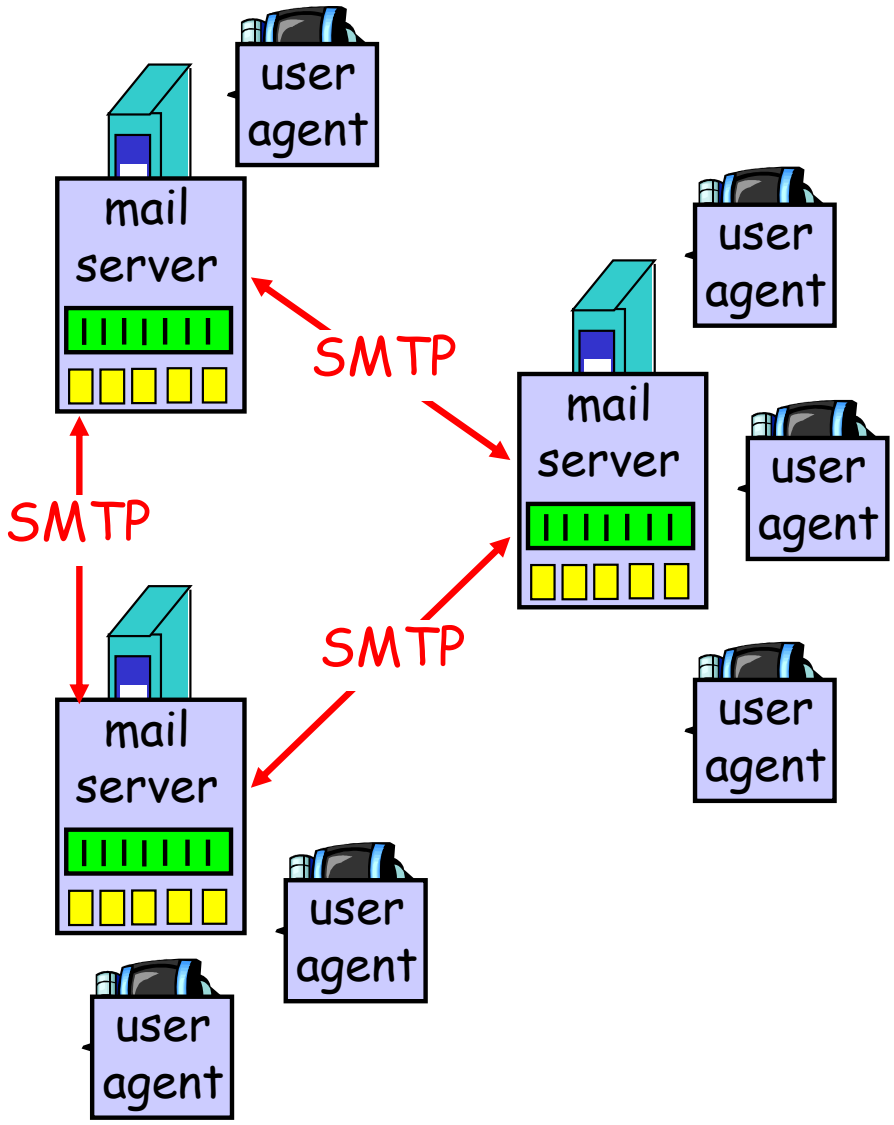
- a.k.a. "mail reader"
- composing, editing, reading mail messages
- e.g., Thunderbird, MS Outlook, Windows Mail , eM Client, The Bat! , Front, Mailbird, Postbox
- outgoing, incoming messages stored on server



Electronic Mail: mail servers

Mail Servers

- mailbox contains incoming messages for user
- message queue of outgoing (to be sent) mail messages
- SMTP protocol between mail servers to send email messages
  - client: sending mail server
  - "server": receiving mail server

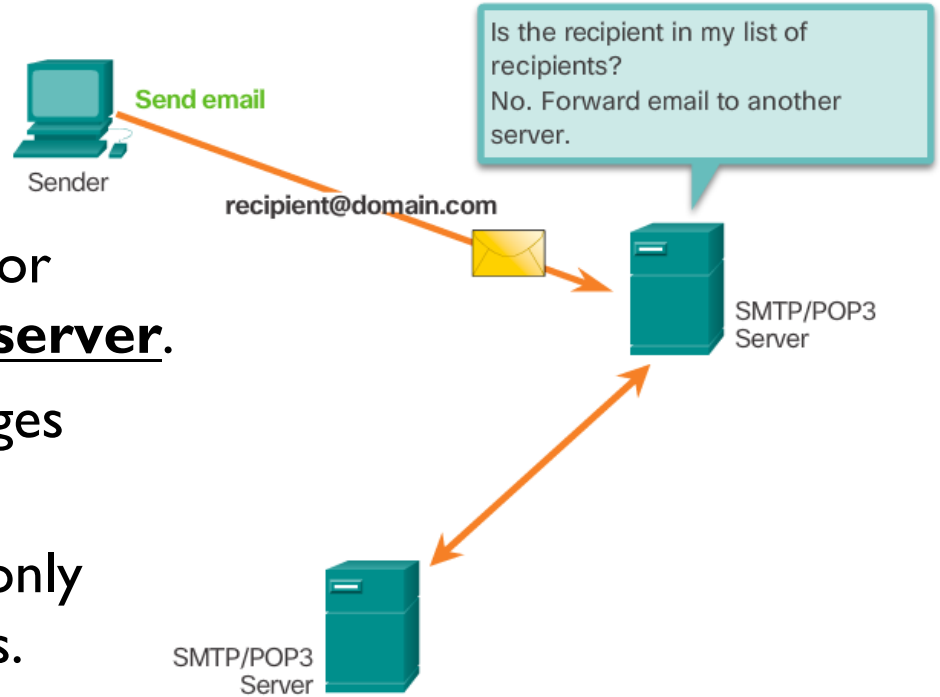


Simple Mail Transfer Protocol (SMTP) [RFC 2821]

- uses TCP to reliably transfer email message
  - from client to server
  - From client server to recipient server
- three phases of transfer
  - handshaking (greeting)
  - transfer of messages
  - closure
- command/response interaction
  - commands: ASCII text
  - response: status code and phrase
- messages must be in 7-bit ASCII !

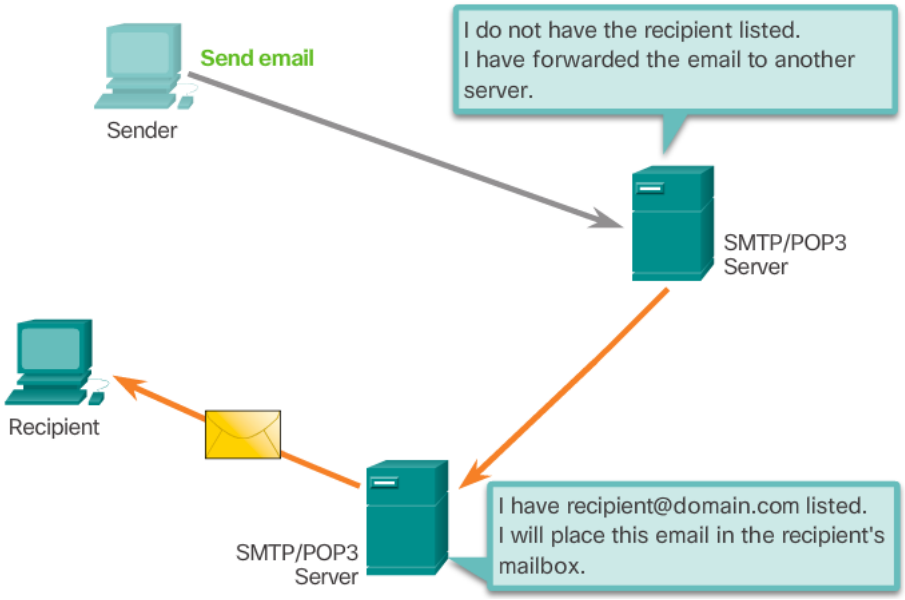
SMTP Operation

- SMTP message formats require a message header and body.
- The body can contain any amount of text.
- The header must have a properly formatted recipient email address and a sender address.
- An SMTP client sends an email by connecting to a SMTP server on port 25.
- The server receives the message and **<u>stores it message in a local mailbox</u>** or **<u>relays the message to another mail server</u>**.
- Users use email clients to retrieve messages stored on the server.
- IMAP and POP are two protocols commonly used by email clients to retrieve messages.



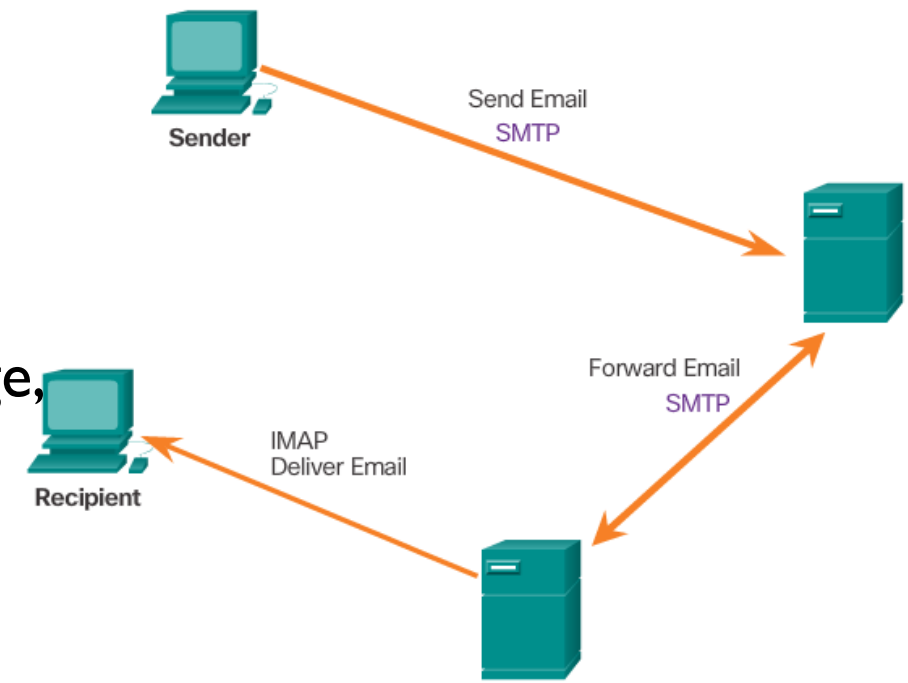
POP Operation

- **Messages are downloaded** from the server to the client.
- The server listens on port 110 TCP for client requests.
- Email clients direct their POP requests to mail servers on port TCP 110.
- The POP client and server exchange commands and responses until the connection is closed or aborted.
- POP allows for email messages to be downloaded to the client's device (computer or phone) and removed from the server.
- There is no centralized location where email messages are kept.
- A downloaded message resides on the device that triggered the download.



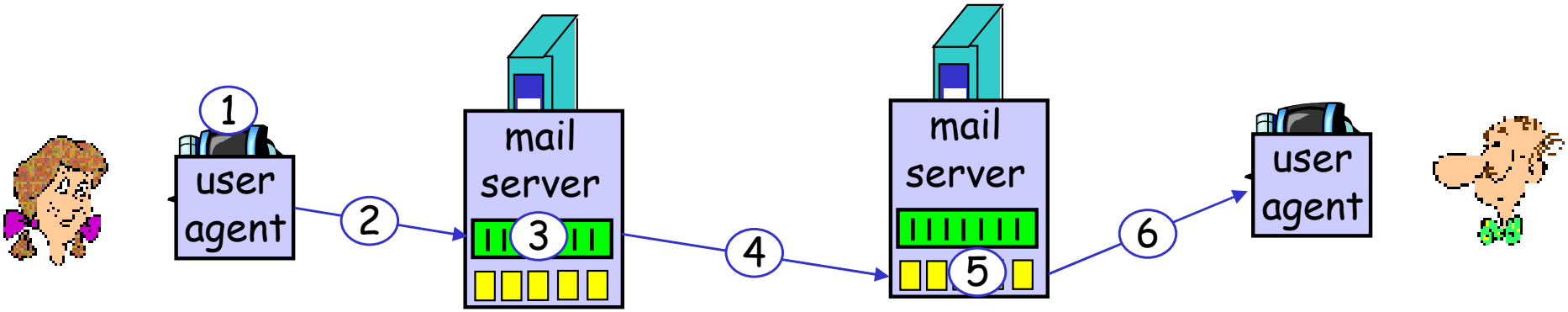
IMAP Operation

- IMAP is another protocol used to retrieve email messages.
- Allows for **messages to be displayed** to the user rather than downloaded.
- The original messages reside on the server until manually deleted by the user.
- Users view copies of the messages in their email client software.
- Users can create a folder hierarchy on the server to organize and store mail.
- That file structure is displayed on the email client.
- When a user decides to delete a message, the server synchronizes that action and deletes the message from the server.

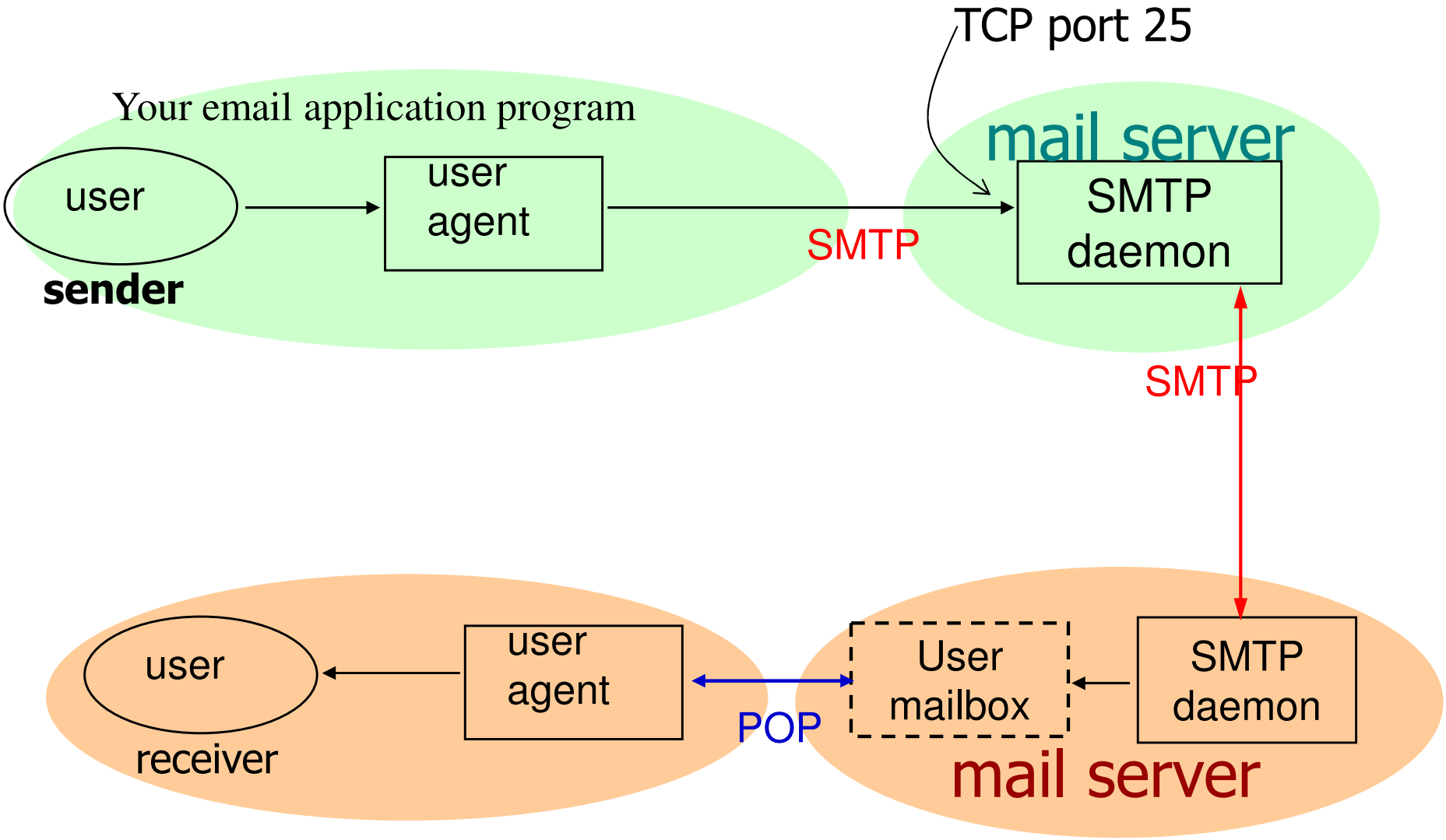


Scenario: Alice sends message to Bob

1) Alice uses UA to compose message and "to" `bob@someschool.edu`
2) Alice's UA sends message to her mail server; message placed in message queue
3) Client side of SMTP opens TCP connection with Bob's mail server
4) SMTP client sends Alice's message over the TCP connection
5) Bob's mail server places the message in Bob's mailbox
6) Bob invokes his user agent to read message

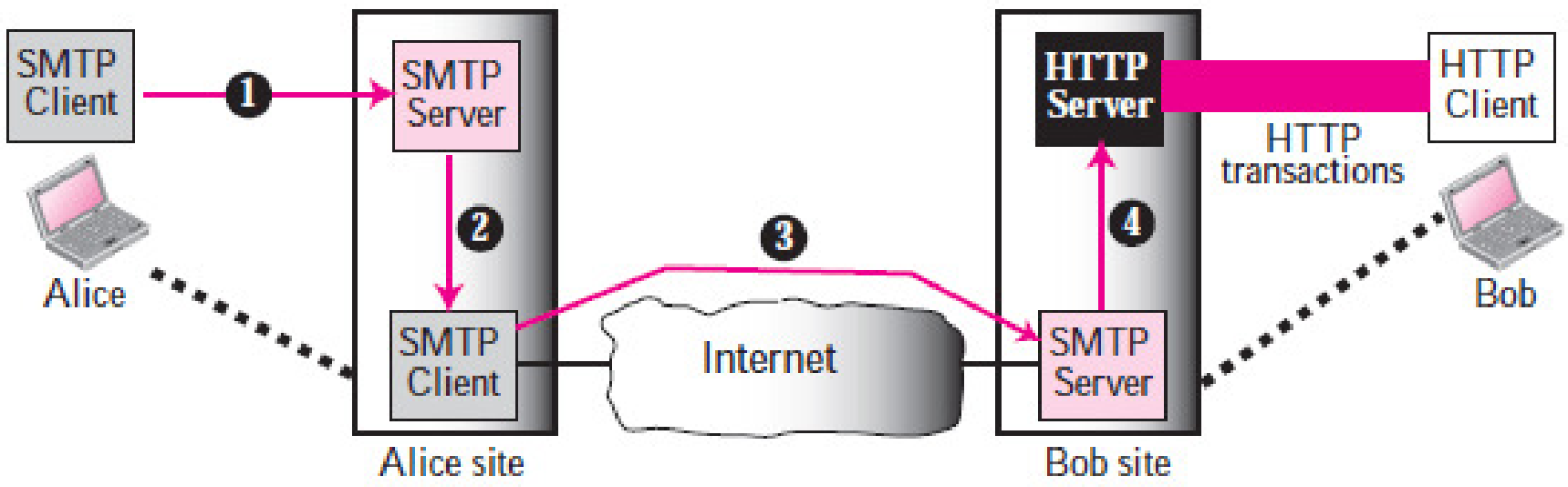


Email delivery



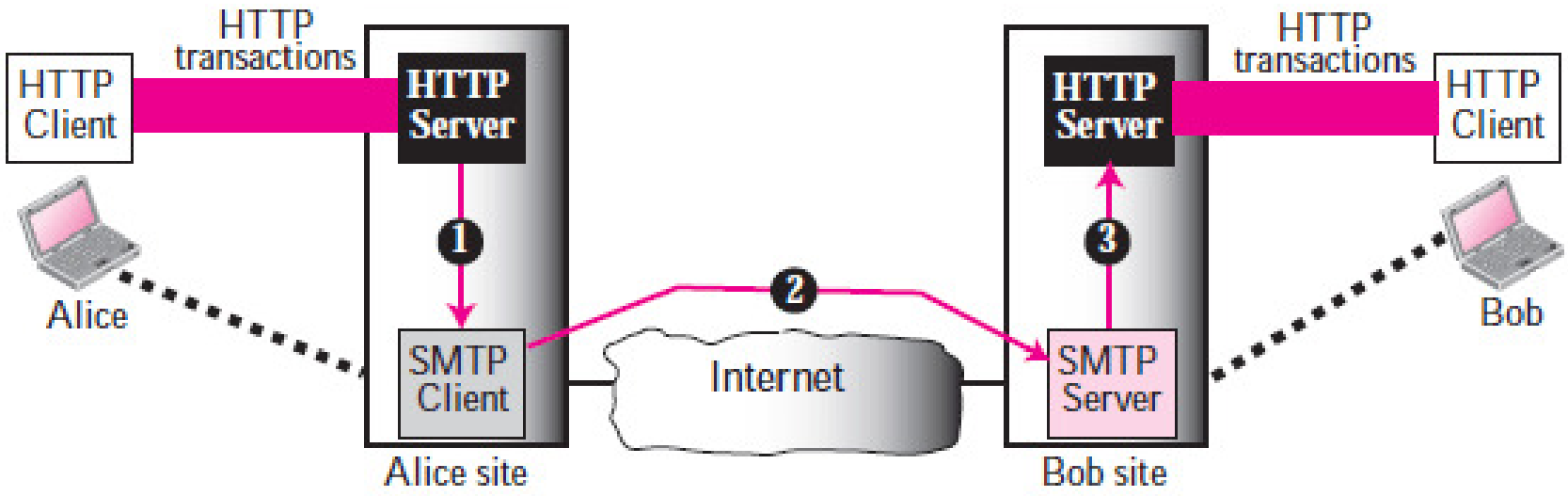
Web Mail / Web based Email

Web-base e-mail, case I



Web Mail / Web based Email

Web-based e-mail, case II



Email: Secure Protocols

- **PGP (Pretty Good Privacy)**
- **Secure/Multipurpose Internet Mail Extensions (S/MIME)**
- **Privacy-Enhanced Mail (PEM)**

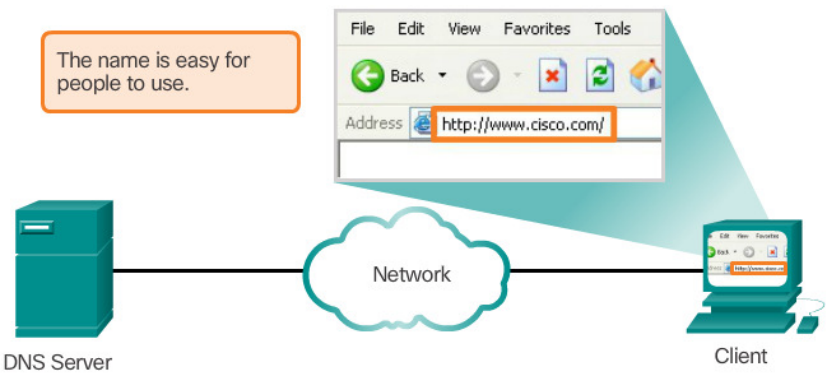
PGP-SMIME-PEM

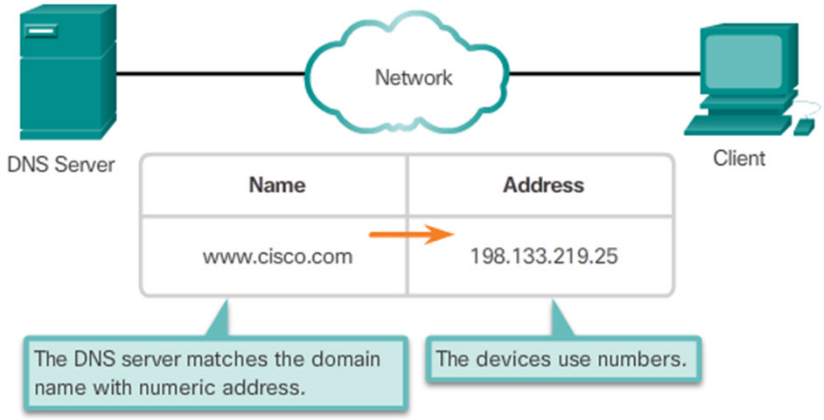
	C	I	A
PGP: It incorporates mechanisms for authentication, confidentiality, compression, e-mail compatibility and segmentation & reassembly . MAIL & FILE	Symmetric encryption-CAST-128, 3-DES, IDEA	SHA	DSS+ SHA or RSA+SHA
SMIME: S/MIME provides the functionality of Enveloped data, signed data, clear signed data and signed and enveloped data. MIME	Diffe-Hellman (Key Exchange) Triple-DES or RC2/40	SHA-1/MD5	SHA-1/MD5 + DSS/RSA
PEM: Mechanism of key management for authentication purposes. Text Based	DES	MD2/MD5	DES+MD5

DNS

Domain Name Service

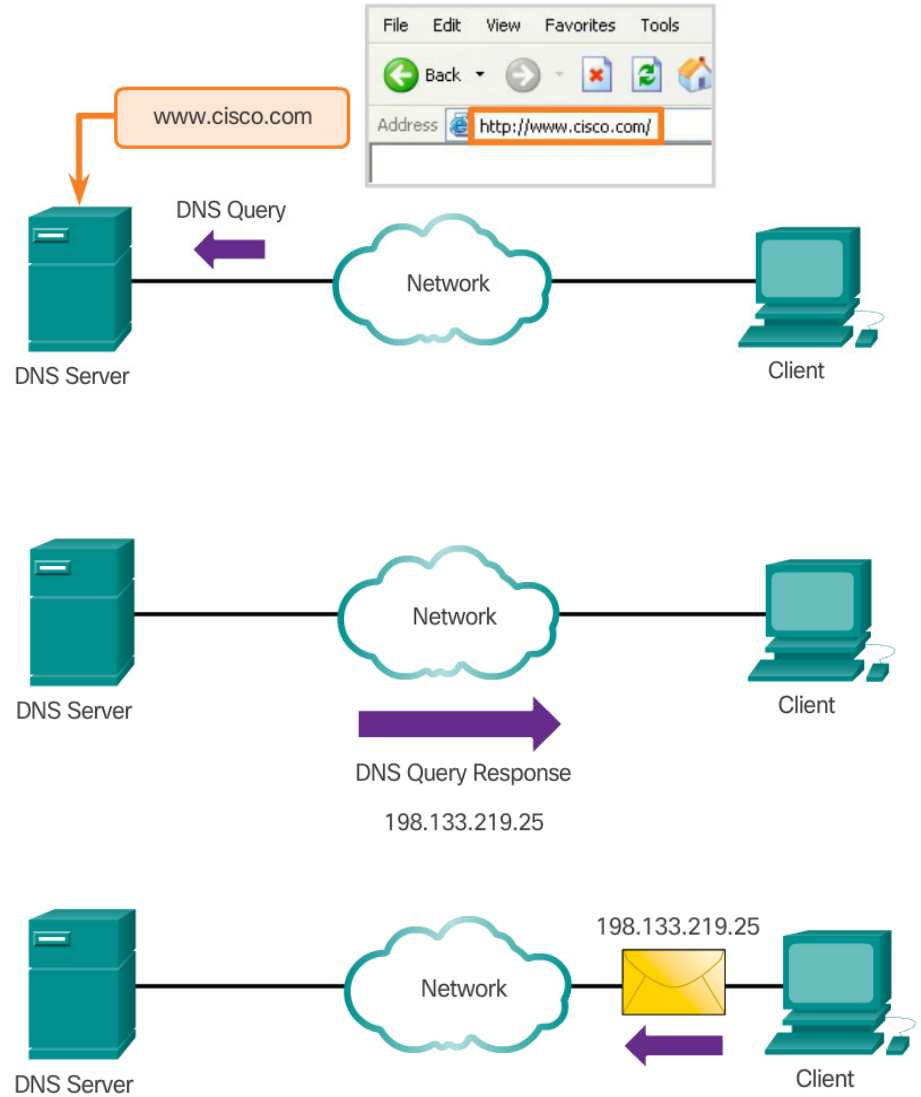
- While IP addresses are crucial for network communication, they are not easy to memorize.
- Domain names are created to make server addresses more user-friendly.
- Domain names such as http://www.cisco.com are user-friendly addresses associated with the IP address of a specific server.
- However, computers still need the actual numeric address before they can communicate.





Domain Name Service (cont.)

- The DNS protocol allows for the dynamic translation of a domain name into the correct IP address.
- The DNS protocol communications using a single format called a message.

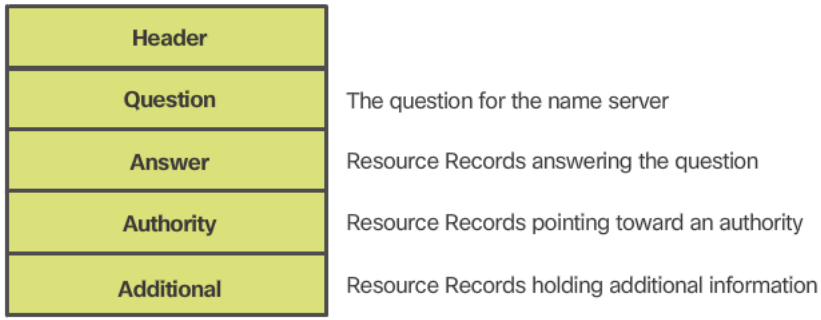


DNS Message Format

- DNS supports different types of records. Some of these record types are:
  - **A** - An end device IPv4 address
  - **NS** - An authoritative name server
  - **AAAA** - An end device IPv6 address (pronounced quad-A)
  - **MX** - A mail exchange record
- DNS servers will first look at its own records to resolve the name. If the server is unable to resolve the name using its locally stored records, it relays the query to other servers.
- The response is then forwarded to the requesting client.
- The DNS Client service on Windows PCs also stores previously resolved names in memory.
- **ipconfig /displaydns** displays all of the cached DNS entries on Windows.

DNS uses the same message format for:

- all types of client queries and server responses
- error messages
- the transfer of resource record information between servers

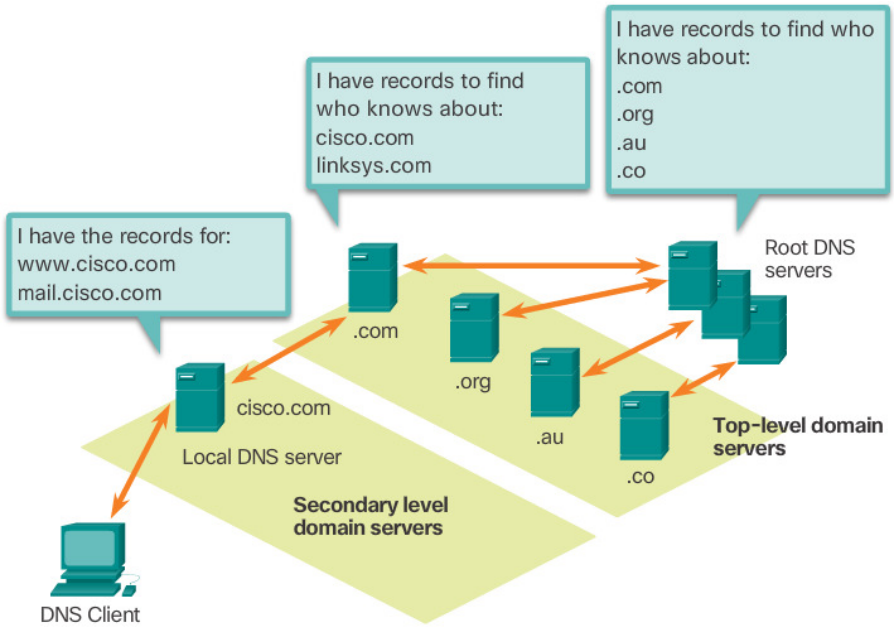


DNS Types: 10 Top DNS Record Types

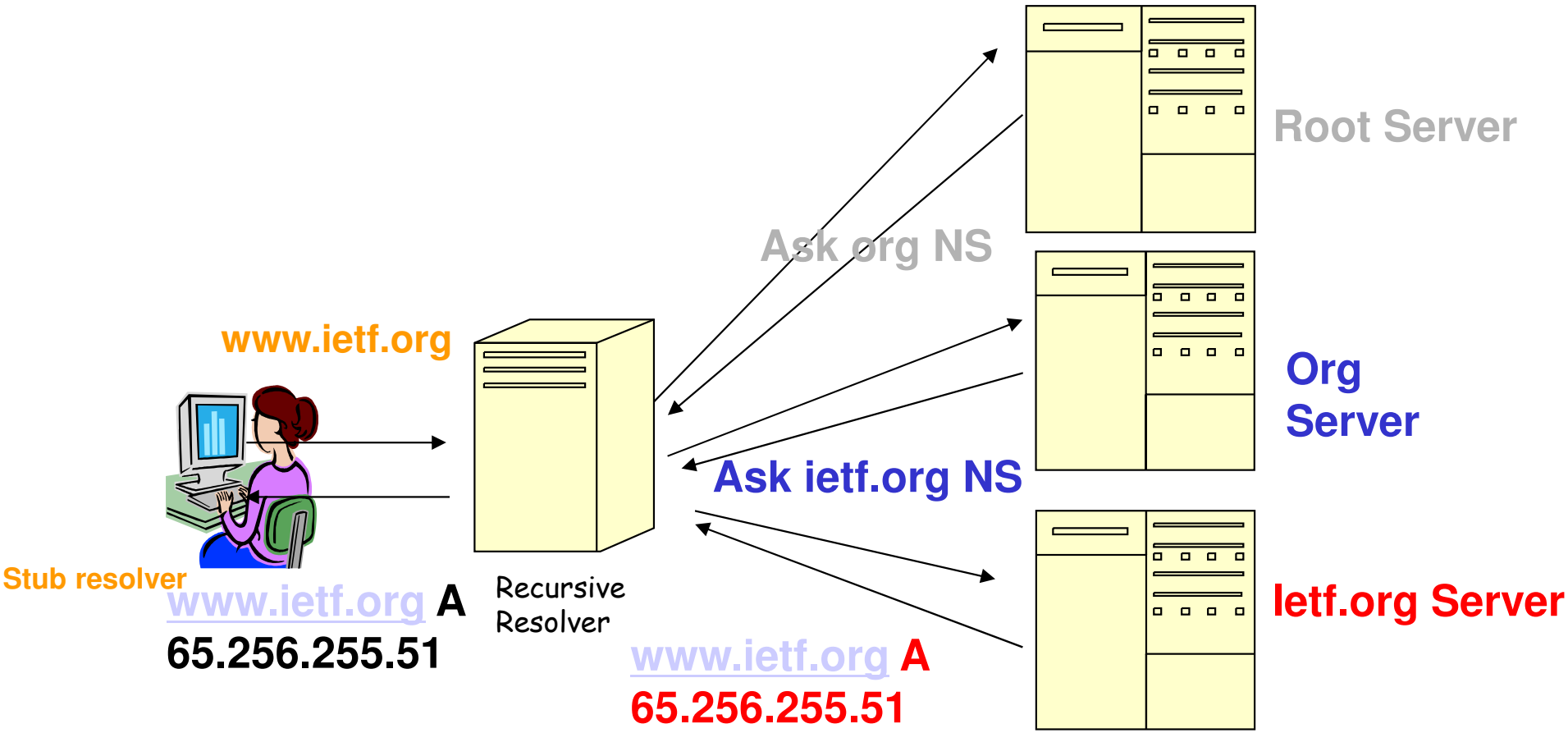
- **Address Mapping record (A Record)—**also known as a DNS host record, stores a hostname and its corresponding IPv4 address.
- **IP Version 6 Address record (AAAA Record)—**stores a hostname and its corresponding IPv6 address.
- **Mail exchanger record (MX Record)—**specifies an SMTP email server for the domain, used to route outgoing emails to an email server.
- **Name Server records (NS Record)—**specifies that a DNS Zone, such as "example.com" is delegated to a specific Authoritative Name Server and provides the address of the name server.
- **Canonical Name record (CNAME Record)—**can be used to alias a hostname to another hostname. When a DNS client requests a record that contains a CNAME, which points to another hostname, the DNS resolution process is repeated with the new hostname.
- **Reverse-lookup Pointer records (PTR Record)—**allows a DNS resolver to provide an IP address and receive a hostname (reverse DNS lookup).
- **Certificate record (CERT Record)—**stores encryption certificates—PKIX, SPKI, PGP, and so on.
- **Service Location (SRV Record)—**a service location record, like MX but for other communication protocols.
- **Text Record (TXT Record)—**typically carries machine-readable data such as opportunistic encryption, sender policy framework, DKIM, DMARC, etc.
- **Start of Authority (SOA Record)—**this record appears at the beginning of a DNS zone file, and indicates the Authoritative Name Server for the current DNS zone, contact details for the domain administrator, domain serial number, and information on how frequently DNS information for this zone should be refreshed.

DNS Hierarchy

- The DNS protocol uses a hierarchical system, with the root at the top and branches below. The naming structure is broken down into small, manageable zones.
- Each DNS server is only responsible for managing name-to-IP mappings for that small portion of the DNS structure.
- Requests for zones not stored in a specific DNS server are forwarded to other servers for translation.
- Top-level domains represent either the type of domain or the country of origin. Examples of top-level domains are:
  - **.com** - a business or industry
  - **.org** - a non-profit organization
  - **.au** - Australia
  - **.co** - Colombia

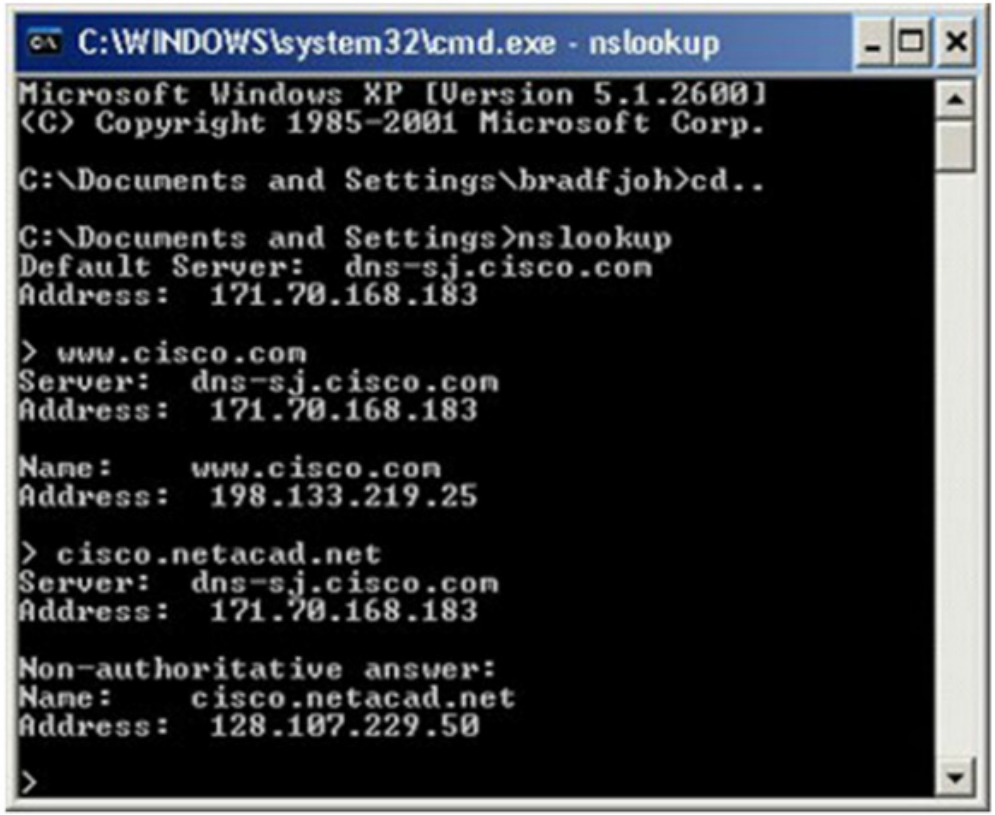


DNS query example



The nslookup Command

- Allows the user to manually place DNS queries.
- It can also be used to troubleshoot name resolution issues.
- Has many options available for extensive testing and verification of the DNS process.

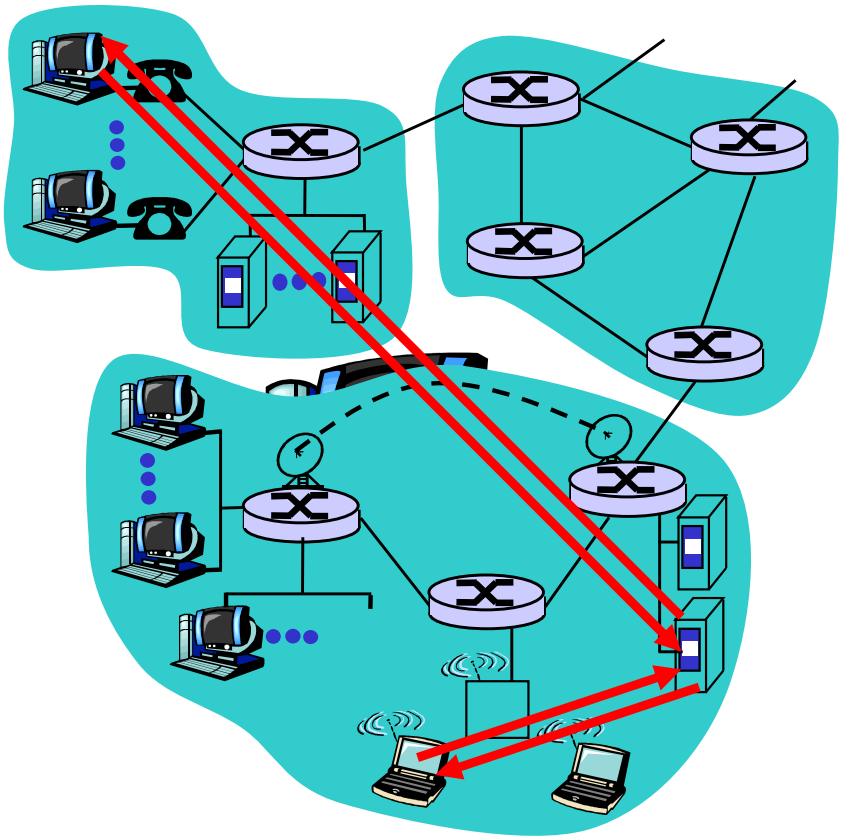


Application architectures

Application architectures

- Client-server
- Peer-to-peer (P2P)
- Hybrid of client-server and P2P

Client-server architecture



server:

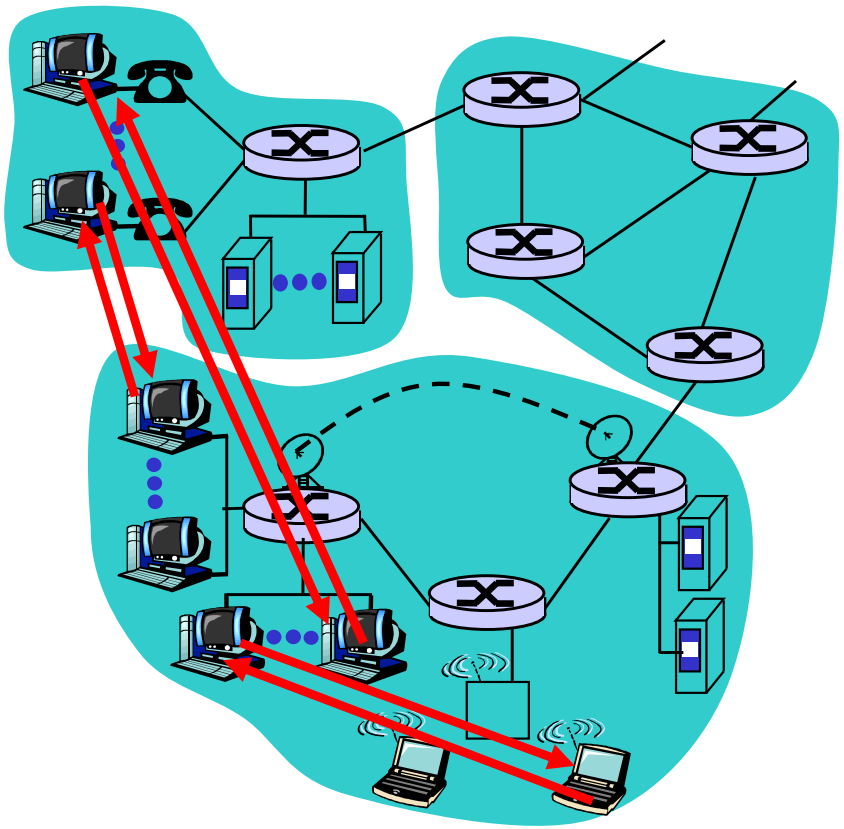
- always-on host
- permanent IP address
- server farms for scaling

clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

Pure P2P architecture

- no always-on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses
- example: Gnutella



Highly scalable

But difficult to manage

Hybrid of client-server and P2P

Napster

- File transfer P2P
- File search centralized:
  - Peers register content at central server
  - Peers query same central server to locate content

Instant messaging

- Chatting between two users is P2P
- Presence detection/location centralized:
  - User registers its IP address with central server when it comes online
  - User contacts central server to find IP addresses of buddies

P2P file sharing

P2P file sharing

Example

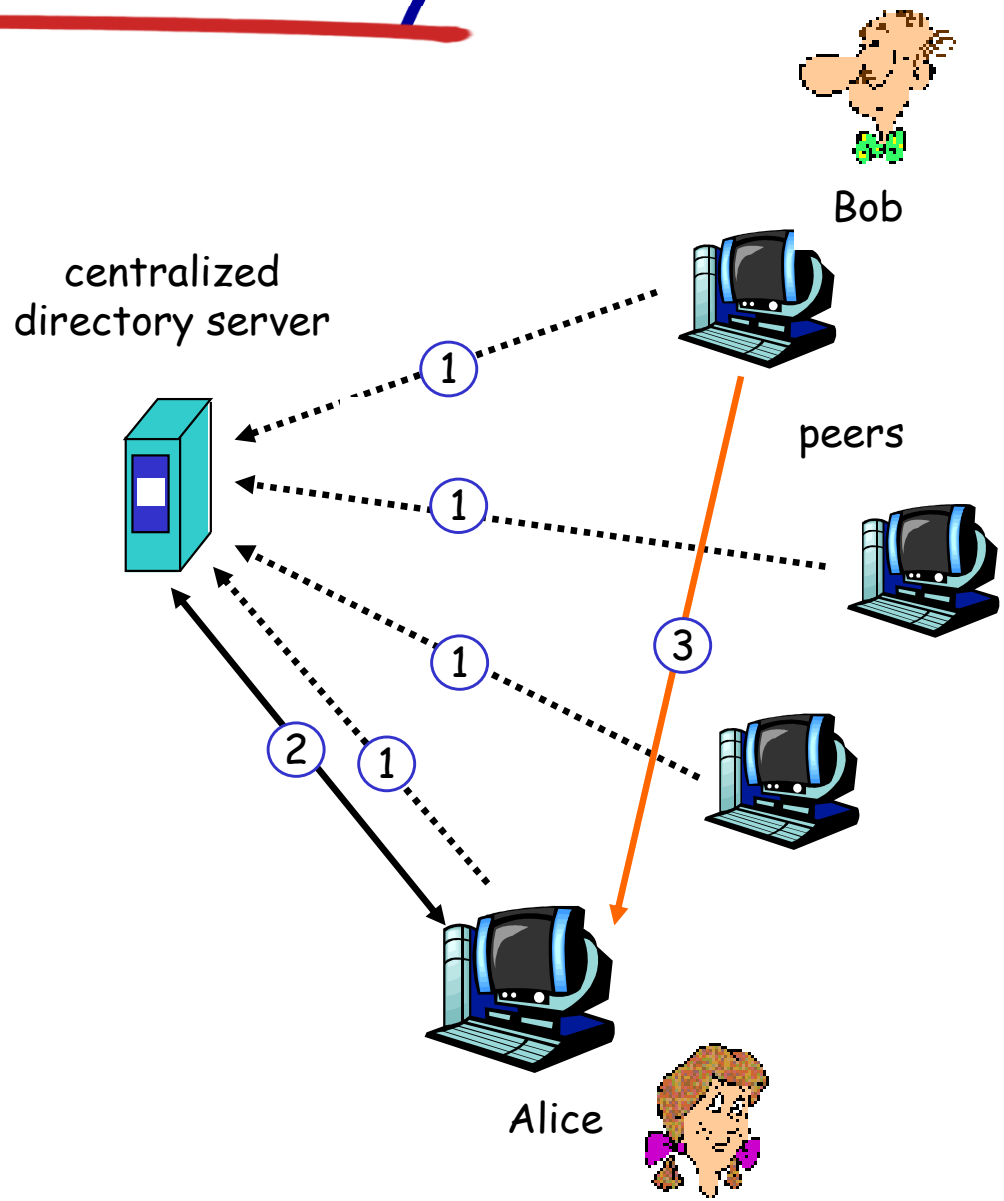
- Alice runs P2P client application on her notebook computer
- Intermittently connects to Internet; gets new IP address for each connection
- Asks for "Hey Jude"
- Application displays other peers that have copy of Hey Jude.
- Alice chooses one of the peers, Bob.
- File is copied from Bob's PC to Alice's notebook: HTTP
- While Alice downloads, other users downloads/uploads to/from Alice.
- Alice's peer is both a Web client and a transient Web server.

All peers are servers = highly scalable!

P2P: centralized directory

original "Napster" design

1) when peer connects, it informs central server:
   - IP address
   - content
2) Alice queries for "Hey Jude"
3) Alice requests file from Bob



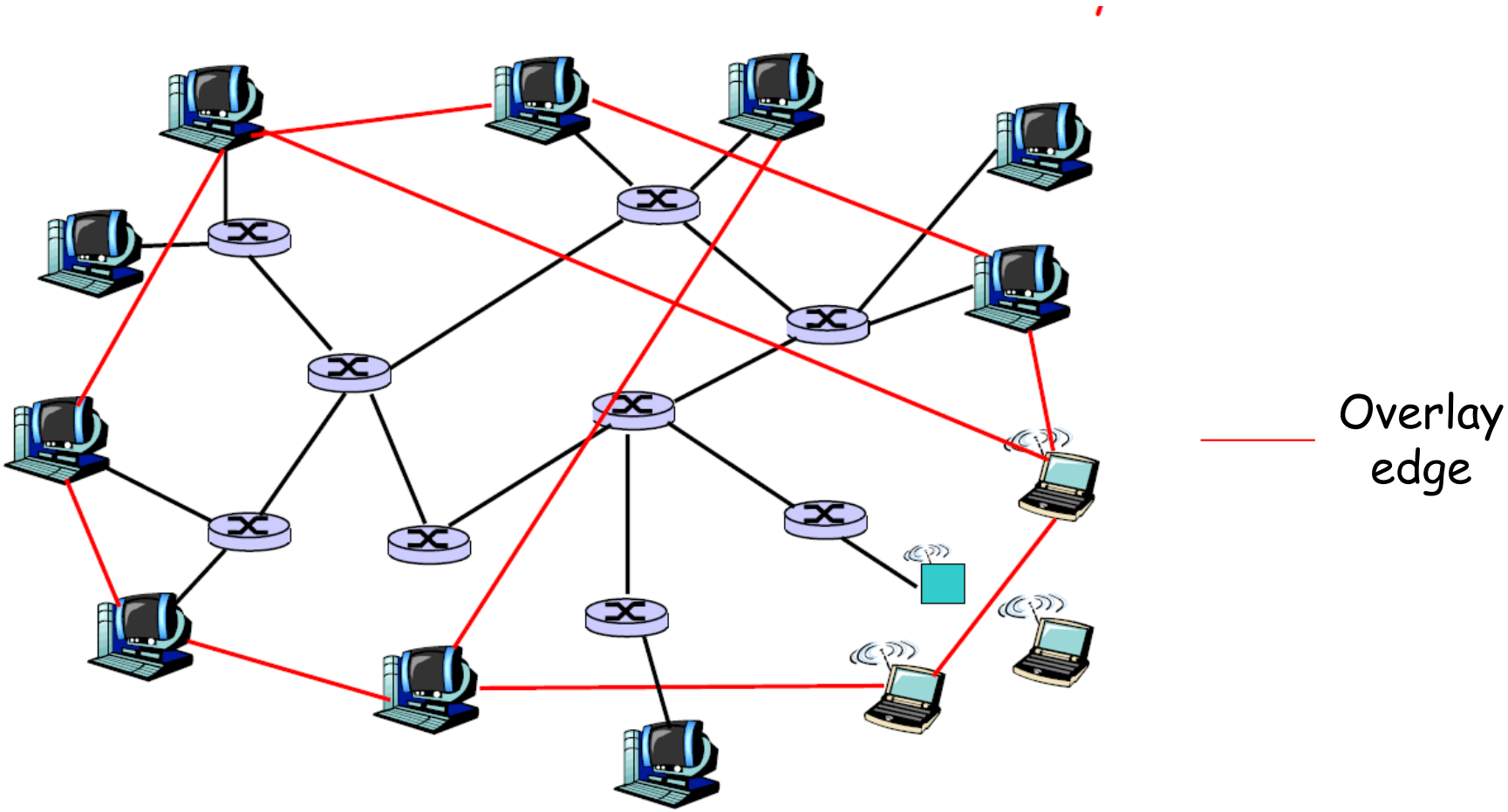
P2P: problems with centralized directory

- Single point of failure
- Performance bottleneck
- Copyright infringement

file transfer is decentralized, but locating content is highly centralized

Overlay Networks

- Network on top of another network



Query flooding: Gnutella

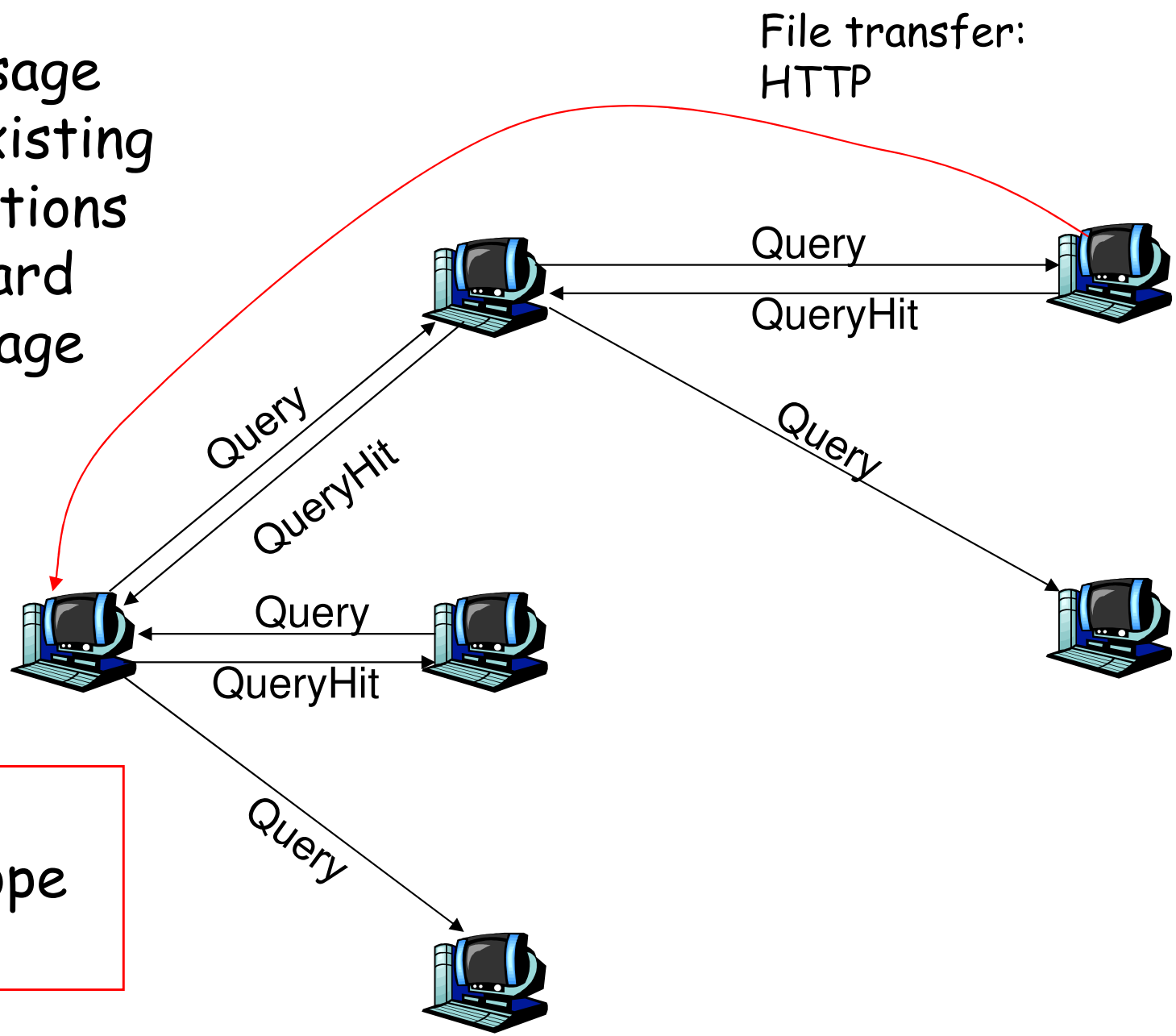
- fully distributed
  - no central server
- public domain protocol
- many Gnutella clients implementing protocol

overlay network: graph

- edge between peer X and Y if there's a TCP connection
- all active peers and edges is overlay net
- Edge is not a physical link
- Given peer will typically be connected with < 10 overlay neighbors

Gnutella: protocol

- Query message sent over existing TCP connections
- peers forward Query message
- QueryHit sent over reverse path



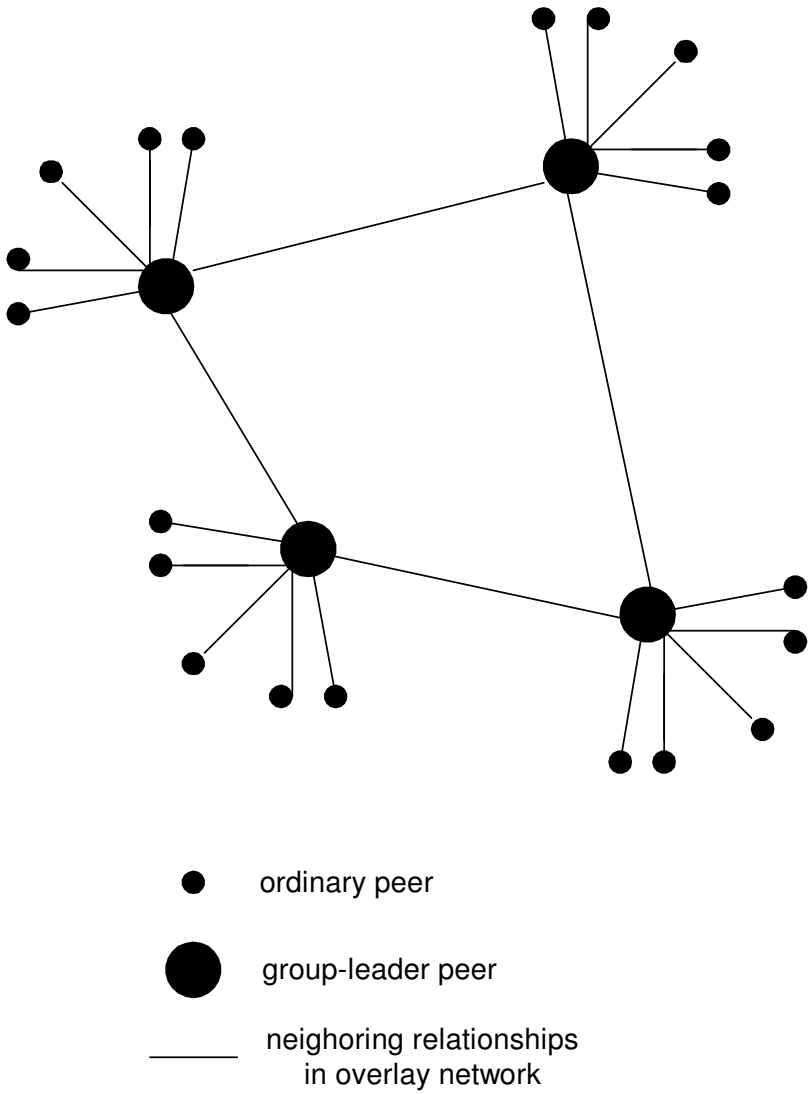
Scalability:
- limited scope
- flooding

Gnutella: Peer joining

1. Joining peer X must find some other peer in Gnutella network: use list of candidate peers
2. X sequentially attempts to make TCP with peers on list until connection setup with Y
3. X sends Ping message to Y; Y forwards Ping message.
4. All peers receiving Ping message respond with Pong message
5. X receives many Pong messages. It can then setup additional TCP connections

Exploiting heterogeneity: KaZaA

- Each peer is either a group leader or assigned to a group leader.
  - TCP connection between peer and its group leader.
  - TCP connections between some pairs of group leaders.
- Group leader tracks the content in all its children.



KaZaA: Querying

- Each file has a hash and a descriptor
- Client sends keyword query to its group leader
- Group leader responds with matches:
  - For each match: metadata, hash, IP address
- If group leader forwards query to other group leaders, they respond with matches
- Client then selects files for downloading
  - HTTP requests using hash as identifier, sent to peers holding desired file

Socket programming with TCP

Socket programming

Goal: learn how to build client/server application that communicate using sockets

Socket API

- introduced in BSD4.1 UNIX, 1981
- explicitly created, used, released by apps
- client/server paradigm
- two types of transport service via socket API:
  - unreliable datagram
  - reliable, byte stream-oriented

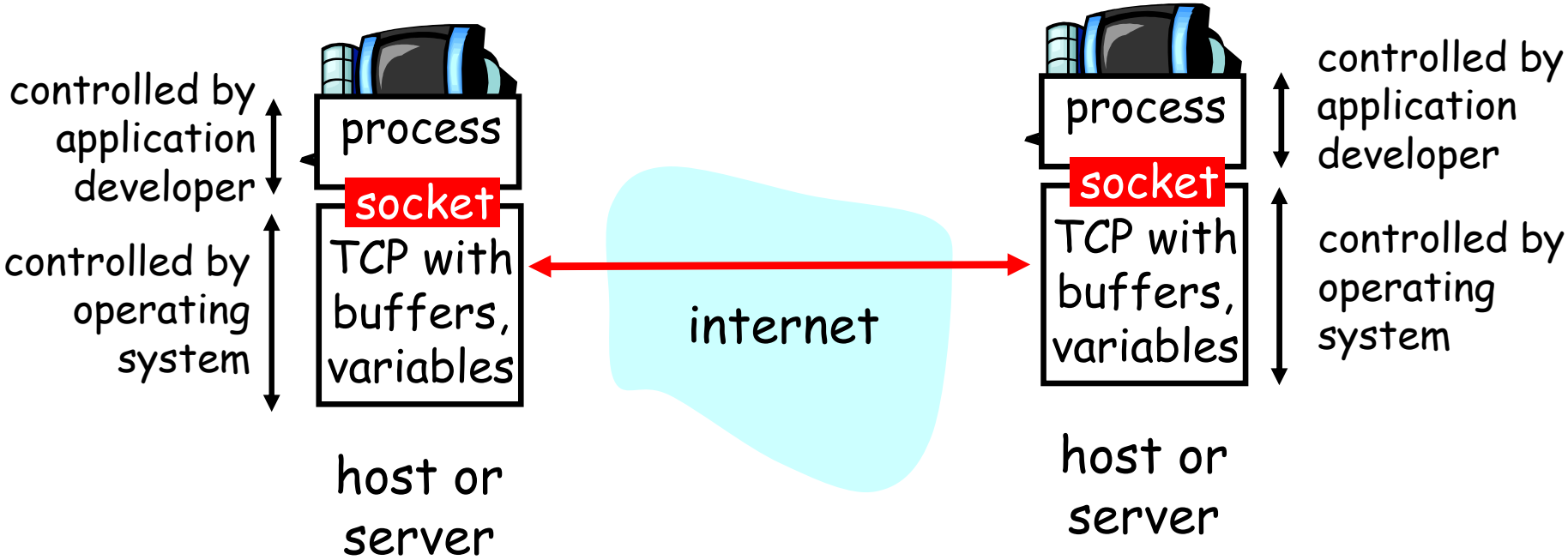
socket

an *OS-controlled* interface (a "door") into which application process can both send and receive messages to/from another application process

Socket-programming using TCP

Socket: a door between application process and end-end-transport protocol (UDP or TCP)

TCP service: reliable transfer of **bytes** from one process to another



Socket programming *with TCP*

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- creating client-local TCP socket
- specifying IP address, port number of server process
- When client creates socket: client TCP establishes connection to server TCP

- When contacted by client, server TCP creates new socket for server process to communicate with client
  - allows server to talk with multiple clients
  - source port numbers used to distinguish clients

application viewpoint

TCP provides reliable, in-order transfer of bytes ("pipe") between client and server

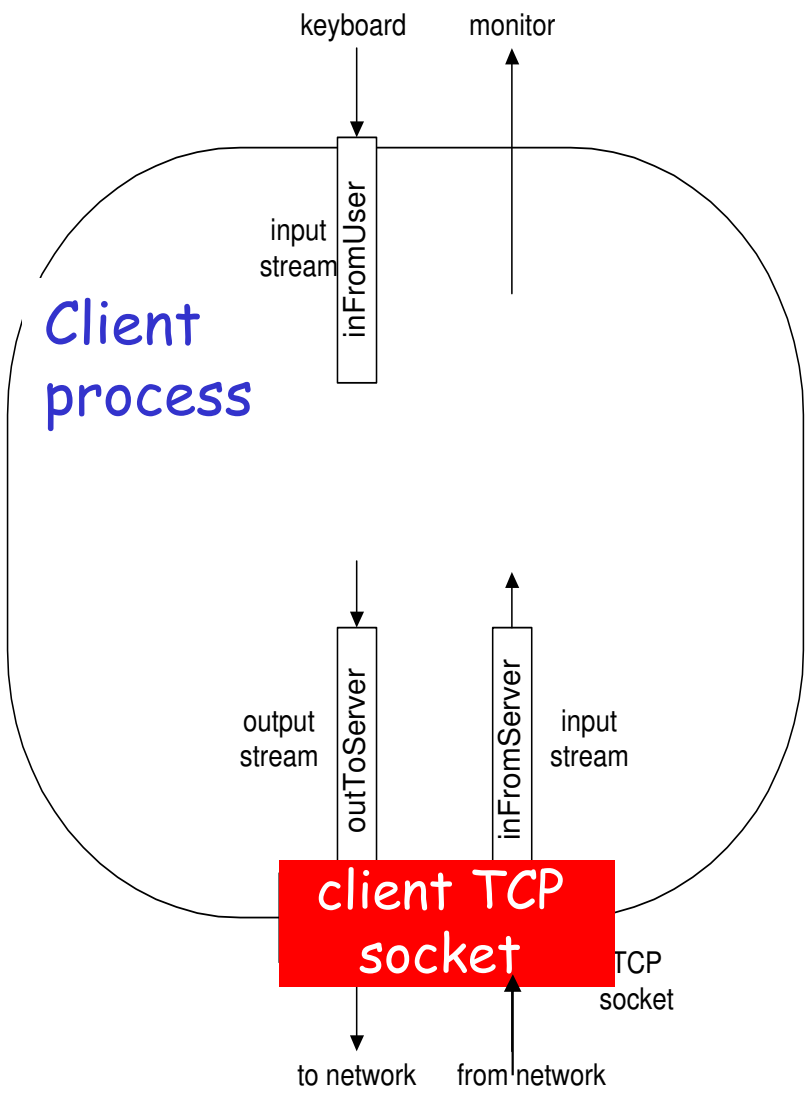
Stream jargons

- A stream is a sequence of characters that flow into or out of a process.
- An input stream is attached to some input source for the process, eg, keyboard or socket.
- An output stream is attached to an output source, eg, monitor or socket.

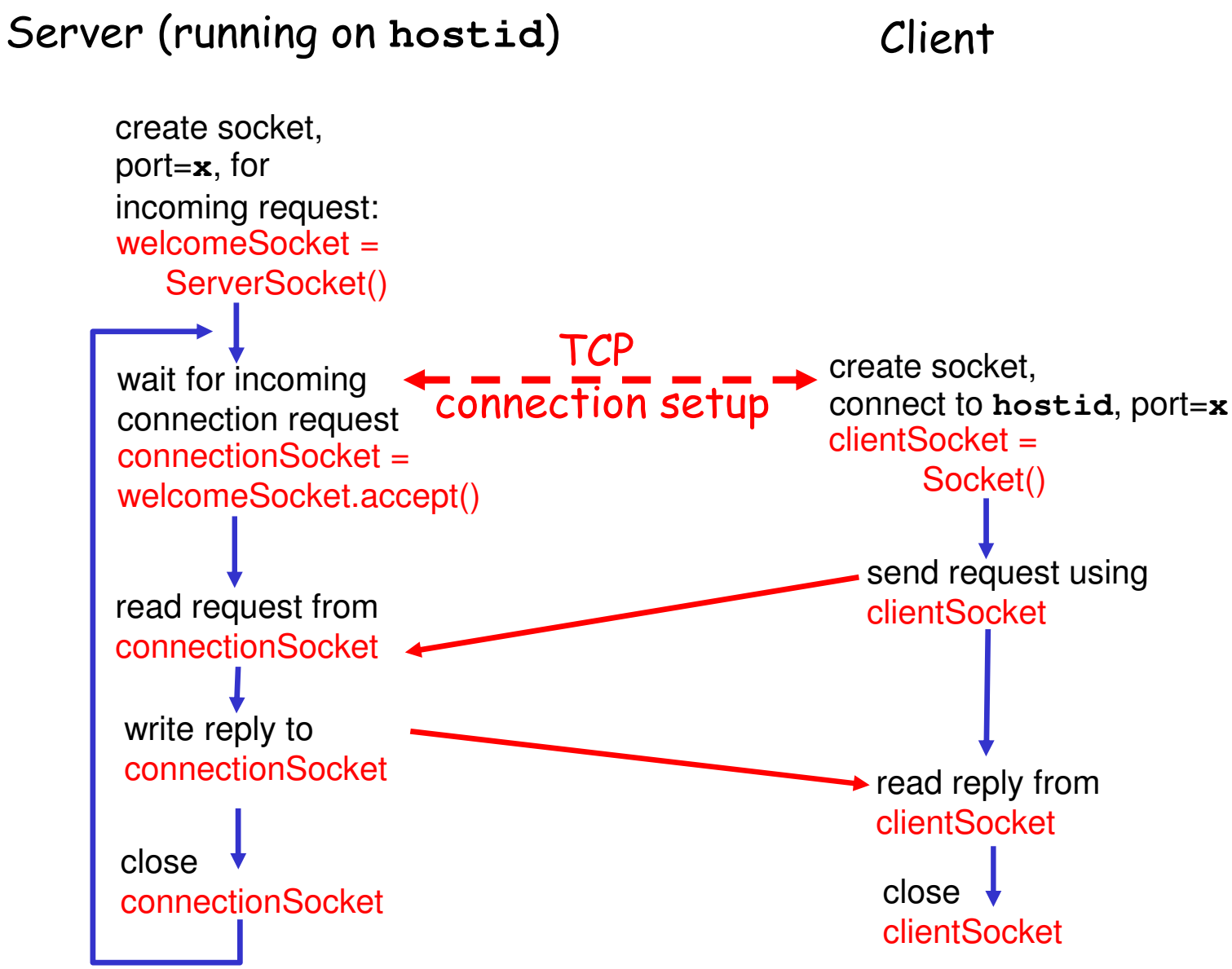
Socket programming with TCP

Example client-server app:

1) client reads line from standard input (**`inFromUser`** stream) , sends to server via socket (**`outToServer`** stream)
2) server reads line from socket
3) server converts line to uppercase, sends back to client
4) client reads, prints modified line from socket (**`inFromServer`** stream)



Client/server socket interaction: TCP



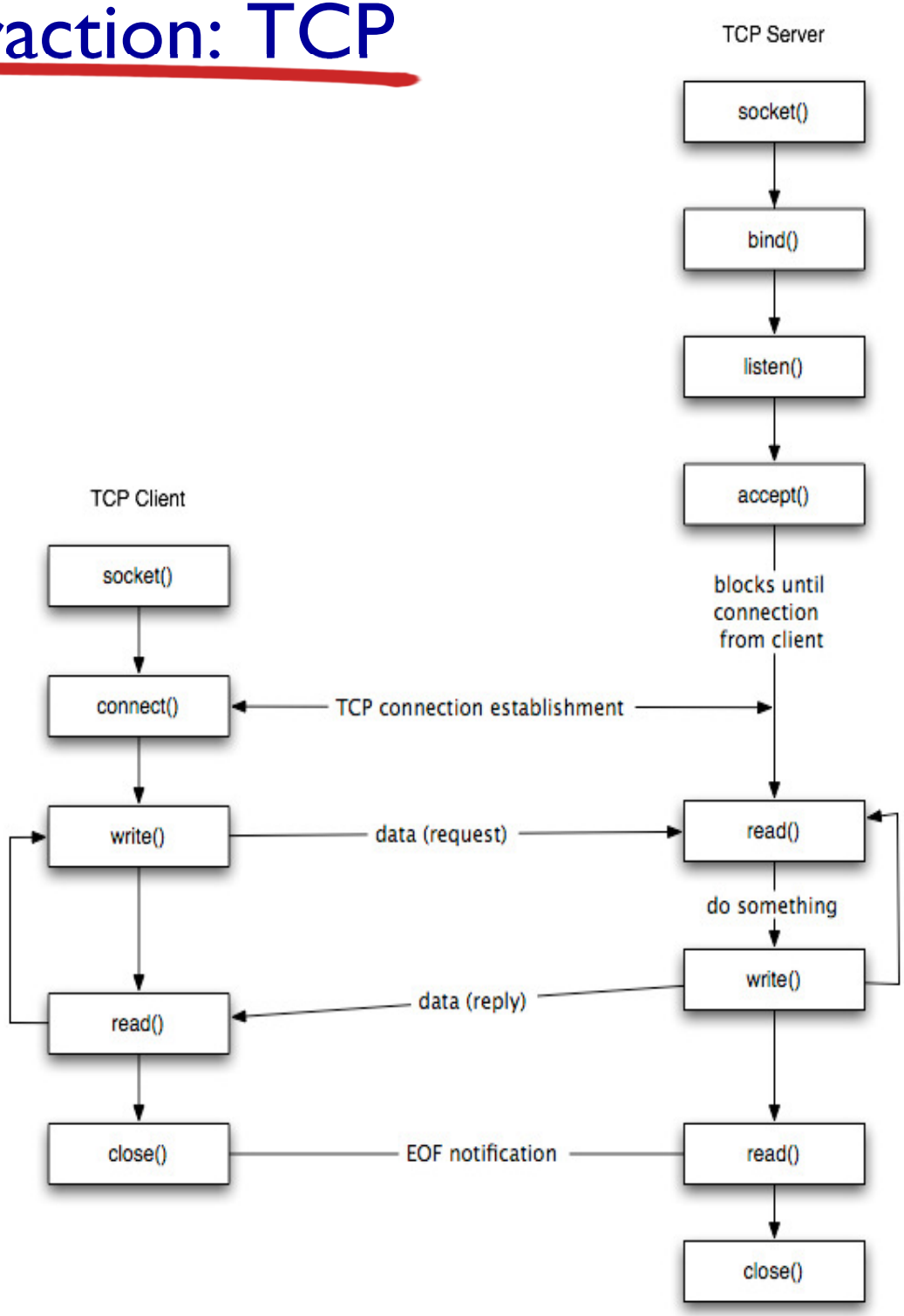
Client/server socket interaction: TCP

As shown in the figure, the steps for establishing a TCP socket on the client side are the following:

- Create a socket using the `socket()` function;
- Connect the socket to the address of the server using the `connect()` function;
- Send and receive data by means of the `read()` and `write()` functions.
- Close the connection by means of the `close()` function.

The steps involved in establishing a TCP socket on the server side are as follows:

- Create a socket with the `socket()` function;
- Bind the socket to an address using the `bind()` function;
- Listen for connections with the `listen()` function;
- Accept a connection with the `accept()` function system call. This call typically blocks until a client connects with the server.
- Send and receive data by means of `send()` and `receive()`.
- Close the connection by means of the `close()` function.



Socket programming *with UDP*

UDP: no "connection" between client and server

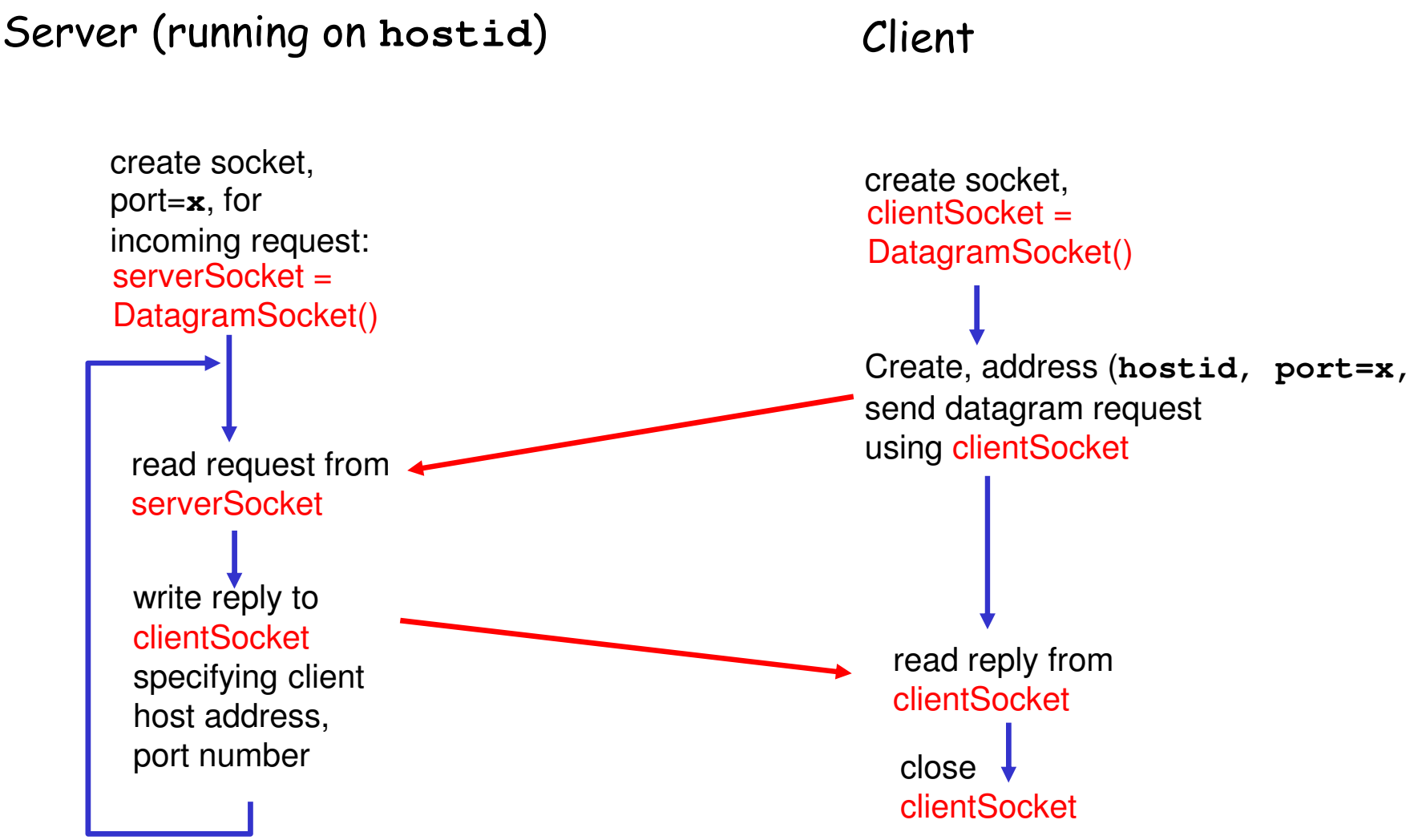
- no handshaking
- sender explicitly attaches IP address and port of destination to each packet
- server must extract IP address, port of sender from received packet

UDP: transmitted data may be received out of order, or lost

application viewpoint

UDP provides unreliable transfer of groups of bytes ("datagrams") between client and server

Client/server socket interaction: UDP



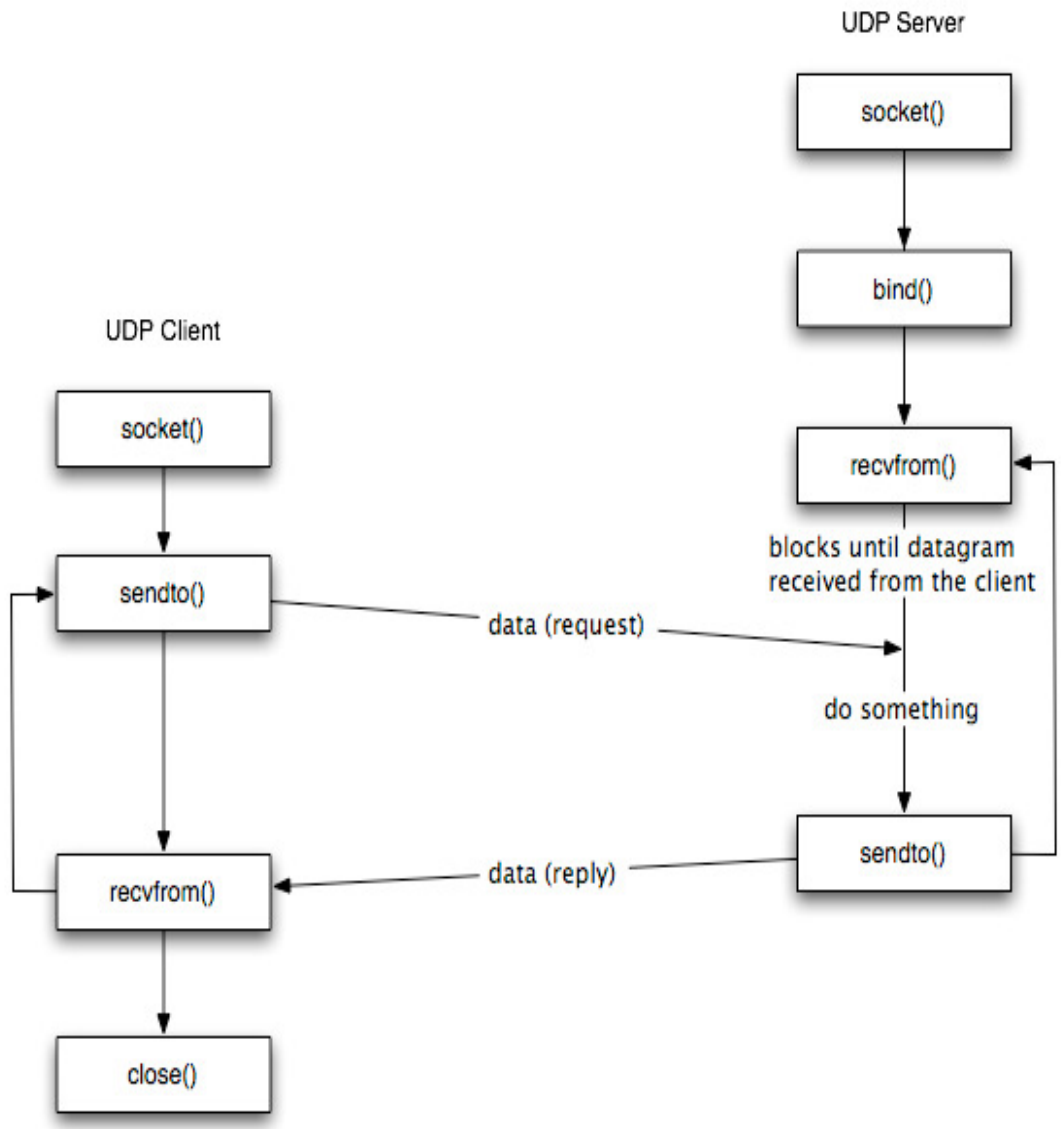
Client/server socket interaction: UDP

As shown in the Figure, the steps of establishing a UDP socket communication on the client side are as follows:

- Create a socket using the socket() function;
- Send and receive data by means of the recvfrom() and sendto() functions.

The steps of establishing a UDP socket communication on the server side are as follows:

- Create a socket with the socket() function;
- Bind the socket to an address using the bind() function;
- Send and receive data by means of recvfrom() and sendto().



Summary

Our study of network apps now complete!

- Application architectures
  - client-server
  - P2P
  - hybrid
- application service requirements:
  - reliability, bandwidth, delay
- Internet transport service model
  - connection-oriented, reliable: TCP
  - unreliable, datagrams: UDP
- specific protocols:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
- socket programming

Summary

Most importantly: learned about *protocols*

- typical request/reply message exchange:
  - client requests info or service
  - server responds with data, status code
- message formats:
  - headers: fields giving info about data
  - data: info being communicated

- centralized vs. decentralized
- stateless vs. stateful
- reliable vs. unreliable msg transfer